

Napredna obdelava slik

(2. letnik RIT – MAG, l. 2025/2026)

izr. prof. dr. Božidar Potočnik

Fakulteta za elektrotehniko, računalništvo in informatiko

Kabinet: G2-2N.37 (33)

Govorilne ure: sredo, 9.00 – 11.00

E-mail: bozidar.potocnik@um.si

Predvideni študijski rezultati in obveznosti študenta

▷ Cilji:

- ★ Vpeljati študente v napredne tehnike obdelave dvo ali več dimenzionalnih digitalnih slik.

▷ Znanje in razumevanje:

- ★ izkazati znanje in razumevanje naprednih postopkov predobdelave in segmentacije slik ter jih analizirati in uporabiti pri načrtovanju rešitev za specifične problemske domene,
- ★ izbrati ustrezno metodo glede na prepoznan segmentacijski problem,
- ★ oceniti in ovrednoti rešitve razvitih tehnik napredne obdelave slik.

▷ ECTS: 6 točk $\approx$ 180 ur dela

▷ Kontaktne ure:

- ★ Predavanja: 30 ur
- ★ Laboratorijske oz. računalniške vaje: 30 ur

▷ Samostojno delo študenta: $\approx$ 120 ur

- ★ Dokončanje laboratorijskih vaj in priprava na pisni izpit

Načini ocenjevanja in razpored predavanj

▷ Načini ocenjevanja:

- ★ Končno oceno tvorijo:
 - ▷ Opravljene laboratorijske vaje – 50 % oz. 500 točk (minimalno 25 % oz. 250 točk)
 - ▷ Pisni izpit – 50 % oz. 500 točk (minimalno 25 % oz. 250 točk)
- ★ **Ne pozabite se prijavljati na izpitne roke!**

▷ Predviden razpored predavanj:

- | | |
|--|---------------------------------------|
| 1. teden: 2.10. | 10. teden: 4.12. |
| 2. teden: 9.10. | 11. teden: 11.12. |
| 3. teden: 16.10. | 12. teden: 18.12. |
| 4. teden: 23.10. | 13. teden: 8.1. |
| 5. teden: 30.10. | 14. teden: 15.1. |
| 6. teden: 6.11. | 15. teden: 22.1. – 2. kolokvij |
| 7. teden: 13.11. | |
| 8. teden: pon, 17.11. – 1. kolokvij | |
| 9. teden: pon, 24.11. | |

Vsebina in literatura

▷ Vsebina:

1. Konvolucijske nevronske mreže
2. Napredni koncepti zajemanja slik
3. Napredna predobdelava slik
4. Napredna segmentacija
5. Slikovne transformacije

▷ Literatura:

- ★ Gradivo iz predavanj in zapiski
- ★ Gradivo iz predavanj in zapiski za predmet Uvod v računalniški vid in razpoznavanje vzorcev
- ★ B. Potočnik, Osnove razpoznavanja vzorcev z nevronskimi mrežami, UM-FERI, 2007.
- ★ (Ostali temeljni študijski viri navedeni v učnem načrtu predmeta – spletna stran FERI)

Kazalo

1. Konvolucijske nevronske mreže	6
2. Napredni koncepti zajemanja slik	34
3. Napredna predobdelava slik	61
4. Napredna segmentacija	84
5. Slikovne transformacije	128

1. Konvolucijske nevronske mreže

Umetni nevron in umetne nevronske mreže

Konvolucijske nevronske mreže

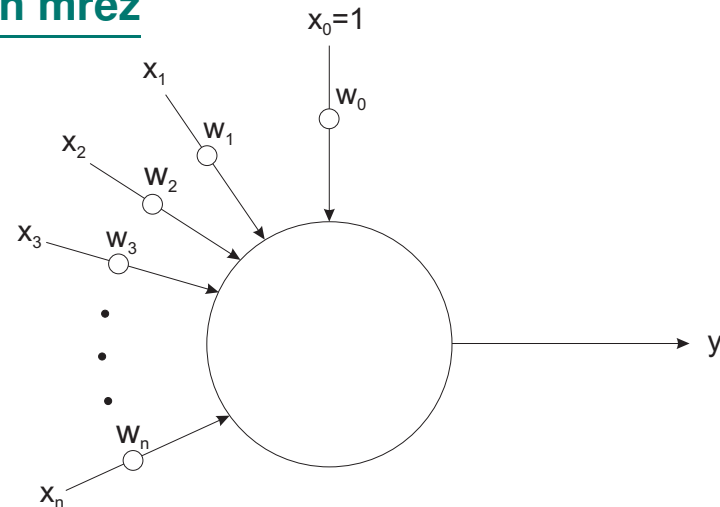
Inženirski pristop k globokim nevronskim mrežam

Aplikacije

Umetni nevron in umetne nevronske mreže

Ponovitev: Nevron – osnovni gradnik nevronskih mrež

- ★ Vhodna vlakna (dendrite) modeliramo z vhodi x_i
- ★ Izhodno vlakno (akson) modeliramo z izhodom y
- ★ Prenosno učinkovitost sinaps modeliramo z realnim številom w_i (tj. utež vhoda)



- ▷ **Obtežen vhod v celično telo** – seštevek električnih impulzov iz vseh dendritov oz. skalarni produkt med vhomom in utežmi

$$s = \sum_{i=1}^n w_i x_i + b \quad \text{oz.} \quad s = \mathbf{w}\mathbf{x} + b \quad \text{oz.} \quad s = \sum_{i=0}^n w_i x_i \quad \text{oz.} \quad s = \hat{\mathbf{w}}\hat{\mathbf{x}}$$

- ★ utež w_0 določa prag (*bias*), v literaturi pogosto označen kot b
- ★ $\mathbf{w} = [w_1, w_2, \dots, w_n]$ oz. $\hat{\mathbf{w}} = [w_0, \mathbf{w}]$
- ★ $\mathbf{x} = [x_1, x_2, \dots, x_n]$ oz. $\hat{\mathbf{x}} = [1, \mathbf{x}]$

- ▷ Izhod y – monotonno naraščajoča funkcija obteženega vhoda s

$$y = f(s)$$

- ★ Funkcija f – **prenosna oz. aktivacijska funkcija nevrona**
- ★ Najbolj uporabljane: ReLU, Leaky ReLU, Maxout, tanh in sigmoidna.

Umetni nevron in umetne nevronske mreže

(2)

ReLU (REctified Linear Unit)

$$f(s) = \max(0, s)$$

- ▷ Danes prva izbira. Pospešuje hitrost učenja NN, enostavna implementacija.

Leaky ReLU

$$f(s) = \max(as, s) \quad a > 0, a \in \mathbb{R}, \quad (\text{npr. } a = 0,01)$$

- ▷ Rešuje problem "umiranja" funkcije ReLU za $s < 0$.
- ▷ Žal izboljšanje zgolj pri nekaterih aplikacijah.

Maxout

$$f(\mathbf{x}) = \max(\mathbf{w}_0\mathbf{x} + b_0, \mathbf{w}_1\mathbf{x} + b_1)$$

- ▷ Aktivacijska funkcija ni klasične oblike $f(s)$, kjer je s obtežen vhod.
- ▷ Odpravlja slabosti funkcij tipa ReLU, a podvoji število parametrov za nevron!

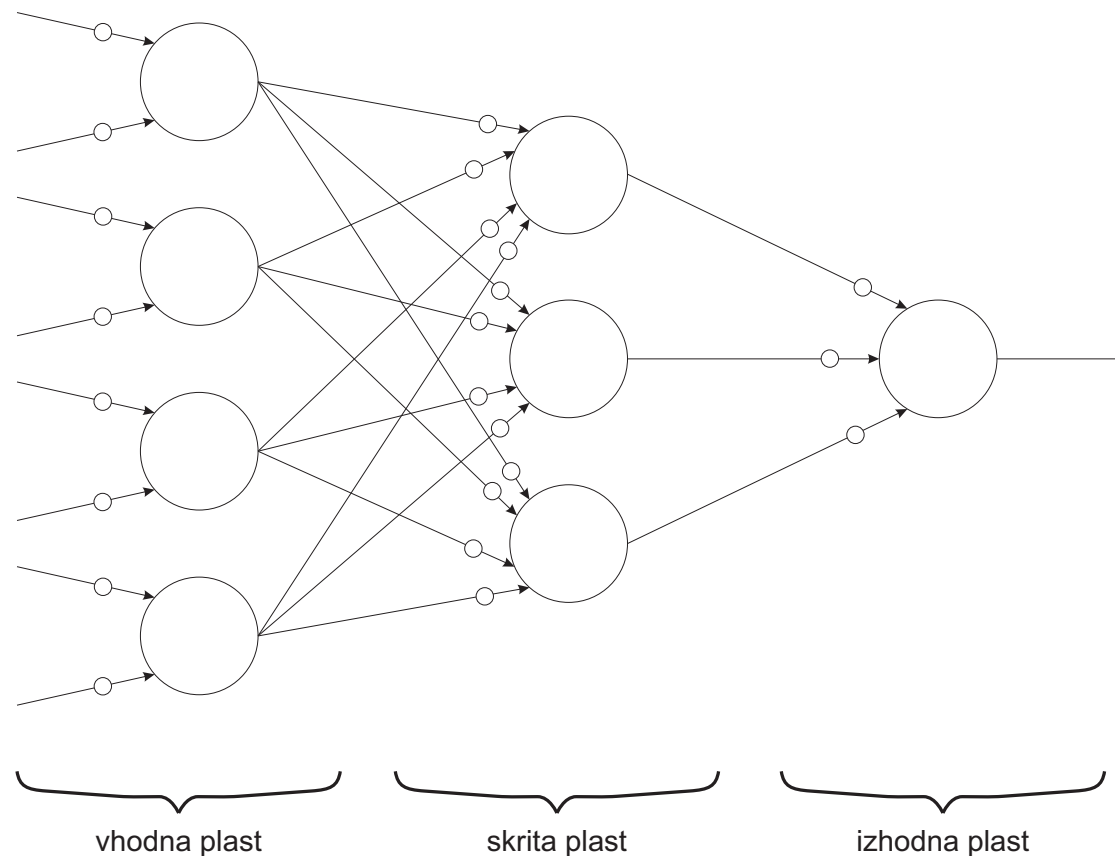
Sigmoidna (*sigmoid, logistic*)

$$f(s) = 1/(1 + e^{-s})$$

- ▷ Med učenjem potisne gradiente v nasičenje ali jih celo "ubije"; ni dobra izbira za vhodno in skrite plasti NN.
- ▷ Funkcija tanh definirana kot: $f(s) = 2 \text{sigmoid}(2s) - 1$

Nevronska mreža (NN) je množica medsebojno povezanih nevronov v graf.

- ▷ **Topologija nevronske mreže** – razmestitev oz. organiziranost nevronov v mreži
- ▷ Nevroni tipično organizirani v **plasti** (*layers*), najpogosteje v **polno povezane plasti** (*fully-connected*).
- ▷ Vrsta NN so **mreže s povezavami naprej** (*feedforward NN*); nevroni so povezani v aciklični graf. Sinonimi: zvezne NN, večplastni zvezni perceptroni.



Umetni nevron in umetne nevronske mreže

(4)

- ▷ Zvezne večplastne perceptrone uporabljamo za identificiranje neznane funkcijske relacije f med dvema množicama $\mathcal{X}$ in $\mathcal{Y}$, pri čemer velja

$$f : \mathcal{X} \mapsto \mathcal{Y}, \quad \mathcal{X} \subset \mathbb{R}^n \wedge \mathcal{Y} \subset \mathbb{R}^m$$

- ▷ Identificiranje poteka na osnovi učne množice $\mathcal{U}_M$ – pari: (vhod, želen izhod):

$$\mathcal{U}_M = \{(\mathbf{x}_i, \mathbf{t}(\mathbf{x}_i)) \mid \mathbf{x}_i \in \mathcal{X} \wedge \mathbf{t}(\mathbf{x}_i) \in \mathcal{Y}\}$$

★ Predpostavimo: $\mathbf{t}(\mathbf{x}_i) = f(\mathbf{x}_i)$

- ▷ Z zveznim večplastnim perceptronom lahko realiziramo neskončno mnogo različnih funkcij $f_{\mathbf{w}}$ (odvisne od vektorja uteži!).
 - ★ Topologija NN določa njeno **kapaciteto** (*capacity*) oz. družino funkcij, ki jih z nevronske mreže lahko realiziramo.
 - ★ Znanje skrito v utežeh nevronov. Vsaka utež je en parameter, ki ga moramo naučiti.
 - ★ Včasih: plitke NN
 - ★ Danes: globoke NN z 20 ali več plastmi; ogromno število parametrov

Učenje nevronske mreže

- ▷ **Nadzorovano učenje** (*supervised learning*): Učimo z učno množico $\mathcal{U}_M$
- ▷ Učenje je iterativni proces.
- ▷ **Korak učenja** (*epoch*) sestoji iz prehoda naprej in prehoda nazaj preko $\mathcal{U}_M$ (ali "mini batch-a"):

Prehod naprej

- ★ Po vstavljanju vhoda $\mathbf{x}_i$, $\mathbf{x}_i \in \mathcal{U}_M$, dobimo na izhodu mreže $\mathbf{y}_i$
- ★ $\forall \mathbf{x}_i \in \mathcal{X} \wedge \mathbf{x}_i \in \mathcal{U}_M$: Izračunamo **izgubo** (*loss*):

$$L_i = L(\mathbf{x}_i, \mathbf{y}_i, \mathbf{t}(\mathbf{x}_i), \mathbf{w})$$

- ★ Izračunamo **strošek** (*cost*):

$$J(\mathbf{w}) = E \left(\sum_i L_i \right), \quad E - \text{matematično upanje}$$

Prehod nazaj

- ★ Določimo gradient stroškovne funkcije, $\nabla_{\mathbf{w}} J(\mathbf{w})$, tipično z **vzvratnim razširjanjem** (*back-propagation*)
- ★ Z učnim algoritmom in izračunanim gradientom $\nabla_{\mathbf{w}} J(\mathbf{w})$ popravimo vse uteži $\mathbf{w}$ v NN.

▷ Stroškovne funkcije

- ★ Navzkrižna entropija (*cross-entropy*) oz. negativna log-verjetnost (*log-likelihood*), srednja kvadratna napaka (*Mean Squared Error – MSE*), srednja absolutna napaka (*Mean Absolute Error – MAE*)
- ★ MSE in MAE vračata slabe rezultate, če uporabljamo optimizacijo na osnovi gradientov.

▷ Učni algoritmi

- ★ Iščejo optimum stroškovne funkcije J v odvisnosti od uteži w nevronske mreže.
- ★ Osnovni algoritmi (fiksna stopnja učenja za vsak parameter): Stohastični gradientni sestop in variante, Moment (*Momentum*)
- ★ Algoritmi z adaptivno stopnjo učenja (za vsak parameter drugačna stopnja učenja): AdaGrad, RMSProp, Adam (*Adaptive Moments*)
- ★ Algoritmi na osnovi približkov gradientov drugega reda: Newtonova metoda, Konjugirani gradienti
- ★ **Izkustven (empiričen) rezultat:** Algoritem RMSProp učinkovit in praktičen optimizacijski algoritem za globoke NN. Danes prva izbira.

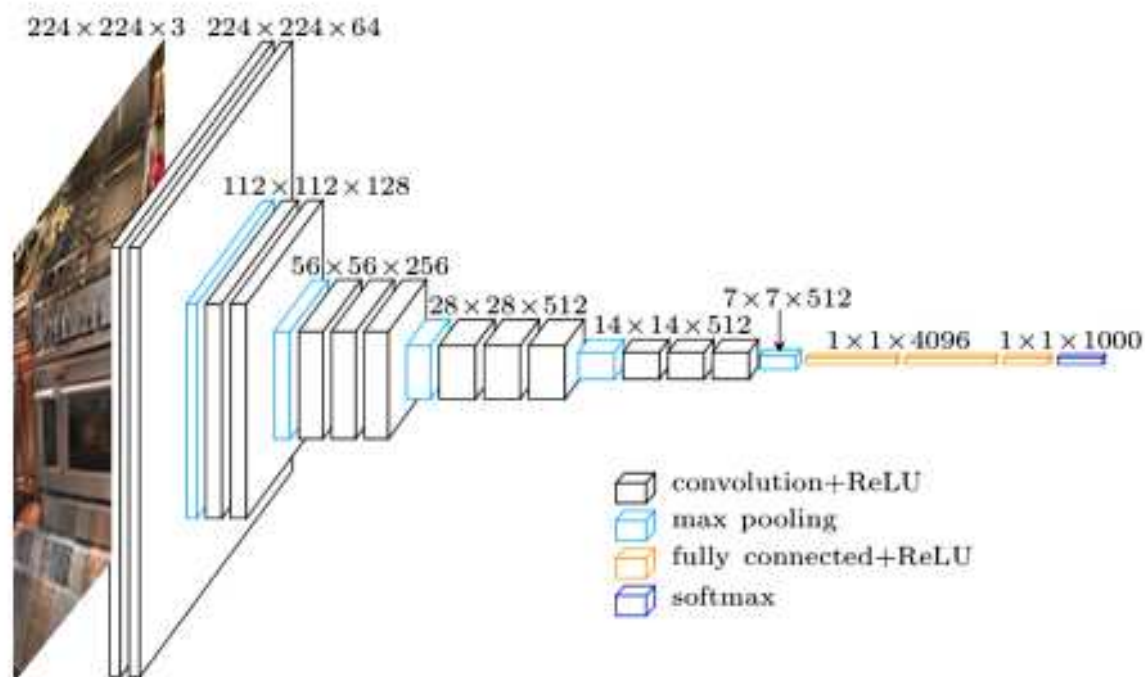
Konvolucijske nevronske mreže

- ▷ Vrsta NN, namenjena obdelovanju podatkov z mreži podobno (*grid-like*) topologijo:
 - ★ Npr.: Časovne serije 1D podatkov, digitalne slike
- ▷ Konvolucijska nevronska mreža (*convolutional NN, CNN*) vedno vključuje vsaj eno operacijo konvolucije.
 - ★ **Ponovitev:** filtriranje signalov in slik, filter oz. jedro oz. maska v 2D, implementacija z obračanjem jedra ali s križno korelacijo
- ▷ Dve različni terminologiji za opis strukture CNN:
 1. Mreža sestoji iz nekaj relativno kompleksnih konvolucijskih plasti, vsaka plast pa ima več faz oz. stopenj (*stage*).
 - ★ Konvolucijska plast tipično iz: konvolucijske faze, sledi faza detekcije oz. aktivacije, na koncu je združevalna faza, nato naslednja konvolucijska plast.
 2. Mreža sestoji iz množice preprostih plasti; vsak obdelovalni korak se torej obravnava kot lastna plast.
 - ★ Tipično sosledje: konvolucijska plast, plast detekcije oz. aktivacije, združevalna plast, naslednja plast.
- ▷ Mreža CNN je običajno globoka NN.

Konvolucijske nevronske mreže

(2)

Primer: VGG mreža s 16 plastmi (vir: [2])

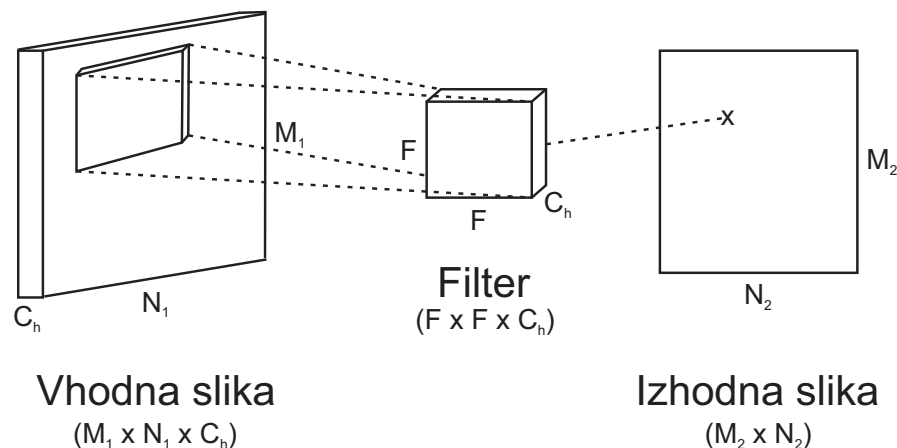


Motivacija za konvolucijske nevronske mreže

- ▷ **Problem:** klasične NN zelo slabo skalirajo na "polno" digitalno sliko
 - ★ Polno povezan nevron v vstopni plasti ima $M \times N \times \text{Št_Kanalov}$ uteži oz. parametrov (+ pragi)
 - ★ Eksplozija števila parametrov pri večjem številu nevronov.
 - ★ **Ogromno število parametrov hitro vodi v nadprileganje (*overfitting*)!**
- ▷ CNN eksplicitno predpostavljajo, da so vhodi digitalne slike.
- ▷ Objekti tipično iz preprostih lokalnih značilnic (npr. robov), ki jih grupiramo v kompleksnejše (npr. obrv).
 - ★ So na različnih delih slike, četudi sestojijo iz enakih preprostih značilnic.
 - ★ **Posledica – Tri pomembne ideje v CNN:**
 1. **Redke interakcije oz. redka povezanost (*sparse interactions*)** – izhodna enota ni povezana z vsako vhodno enoto plasti (majhna jedra!)
 2. **Deljenje parametrov** – uporaba istega parametra (uteži) za več kot eno funkcijo v modelu (učimo en set parametrov za vse lokacije v sliki!)
 3. **"Ekvivariantna" predstavitev oz. ekvivariantnost plasti na translacijo** – ekvivariantnost: če se vhod spremeni, se izhod spremeni na enak način
 - ▷ **Konvolucija ni ekvivariantna na rotacijo in spremembo v skali!**

A. Konvolucijska plast

- ▷ Filtrira vhodno sliko s filtrom.
- ▷ Tipično uporaba **križne korelacije** (skalarni produkt podslike in nezrcaljenega jedra).
- ▷ Jedro premikamo po vrsticah in stolpcih slike s **korakom S** (*stride*).
- ▷ Zaradi ohranjanja dimenzije lahko dodamo k sliki **rob P -tih ničel** (*zero-padding*).



- ▷ **Rezultat:** izhodna slika dimenzije $M_2 \times N_2$, kjer

$$M_2 = (M_1 - F + 2P)/S + 1, \quad N_2 = (N_1 - F + 2P)/S + 1$$

- ▷ **V praksi:** Na sliko apliciramo K različnih filtrov, zato izhod konvolucijskega sloja dimenzije

$$M_2 \times N_2 \times K$$

Konvolucijske nevronske mreže

(5)

- ▷ Uteži filtrov niso poznane vnaprej.
- ▷ Določimo jih v fazi učenja CNN.
- ▷ Vsak filter vpeljuje (ideja deljenja parametrov!)

$$F \times F \times C_h \quad \text{uteži oz. parametrov}$$

- ▷ Skupno potrebujemo za K filtrov

$$(F \times F \times C_h) \times K \quad \text{uteži in } K \text{ pragov}$$

- ▷ Parametri F , S , P in K so **hiperparametri** (*hyperparameters*) CNN.

- ▷ **Tipične nastavitve:**

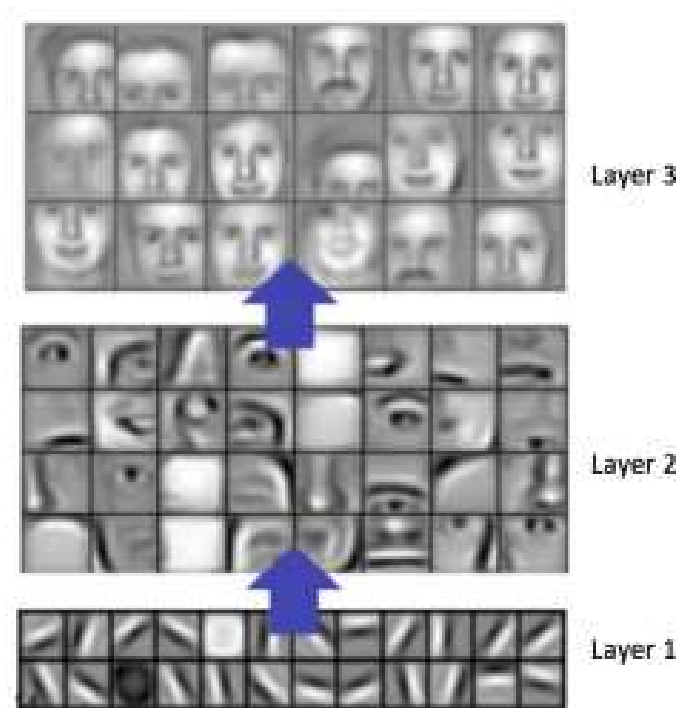
- ★ $F = 3$, $S = 1$ in $P = 1$ – ohranimo dimenzijo vhodne slike
- ★ Če $P = 0$ in/ali $S > 1$ – zmanjšamo dimenzijo vhodne slike
- ★ **Hiperparametre moramo izbrati tako, da sta M_2 in N_2 celi števili!**
- ★ Nastavitev parametrov seveda odvisna od problema, ki ga rešujemo!

Konvolucijske nevronske mreže

(6)

- ▷ V splošnem: Več kot imamo konvolucijskih plasti, bolj kompleksne značilnice se bo zmožna naučiti mreža.
 - ★ nizko nivojske značilnice (*low-level features*) – začetne konvolucijske plasti
 - ★ srednje nivojske značilnice (*mid-level features*) – vmesne konvolucijske plasti
 - ★ visoko nivojske značilnice (*high-level features*) – zaključne konvolucijske plasti

Primer: Naučene značilnice s pomočjo konvolucijske mreže globokega zaupanja (*convolutional deep belief network*) (vir: [3])



B. Plast detekcije oz. aktivacijska plast

- ▷ Aplikira na vhod eno izmed aktivacijskih funkcij (npr. ReLU).
- ▷ Aktivacijo izvajamo na nivoju posameznega elementa.

C. Polno povezana plast (*fully-connected layer*)

- ▷ Vsak nevron v tej plasti je povezan z vsemi vhodi (utež za vsak vhod + prag za nevron).
- ▷ Popolnoma identično kot pri zveznih perceptronih.
- ▷ **Z** ustrezno izbiro hiperparametrov F , P , S in K lahko vsako polno povezano plast transformiramo v konvolucijsko plast!
 - ★ Pri reševanju določenih problemov lahko s tem pohitrimo izvajanje.

D. Združevalna plast (*pooling layer*)

- ▷ Tipično jo vstavljamo med zaporedne konvolucijske plasti.
- ▷ Reducira prostorsko dimenzijo (podvzorčenje!), količino parametrov in računanj v mreži, s tem kontroliramo nadprileganje.
 - ★ Če pretiravamo s podvzorčenjem, povzročimo podprileganje (*underfitting*).
- ▷ Operira neodvisno na vsakem kanalu vhoda (dimenzije $M_1 \times N_1 \times C_h$).

▷ Postopek za en kanal:

- ★ Izberemo velikost filtra $F \times F$ in korak S za vhodno sliko (ničelne obrobe ne dodajamo!).
- ★ Nad okolico $F \times F$ izračunamo vnaprej izbrano funkcijo.
- ★ Izhod je dimenzije $M_2 \times N_2$ (za kanal) oz. $M_2 \times N_2 \times C_h$ (za sliko):

$$M_2 = (M_1 - F)/S + 1, \quad \text{in} \quad N_2 = (N_1 - F)/S + 1.$$

- ## ▷ Združevalne funkcije:
- maksimum v okolici (tipično); povprečje; L^2 -norma; obteženo povprečje, glede na oddaljenost od središča

▷ Nastavitev hiperparametrov v praksi:

- ★ $F = 2$ in $S = 2$ (najpogostejše)
- ★ $F = 3$ in $S = 2$ (prekrivajoče združevanje)

- ## ▷ Združevanje po posameznem kanalu producira invariantnost na translacijo (majhno, znotraj okolice $F \times F$).

- ★ S pomočjo združevanja preko vseh kanalov se lahko značilnice priučijo invariantnosti tudi na druge funkcije (npr. rotacijo).

Konvolucijske nevronske mreže

(9)

Primeri uveljavljenih arhitektur iz področja CNN [4, 9]:

▷ Klasične arhitekture:

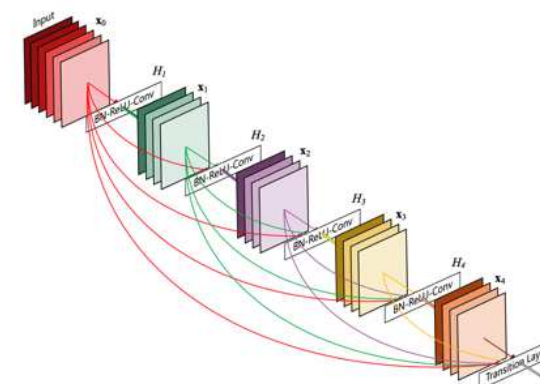
- ★ LeNet (1998)
- ★ AlexNet (2012) – izboljšava LeNet, ima 60M parametrov!
- ★ ZFNet (2013) – izboljšava AlexNet
- ★ VGGNet (2014) – preprosta in homogena struktura, a 140M parametrov

▷ Moderne arhitecture:

- ★ GoogLeNet (2014) – "inception" moduli, manj parametrov kot AlexNet
- ★ ResNet (2015) – blok preostanka (*residual block*), lahko implementira funkcijo identitete
- ★ DenseNet (2016) – na vhod trenutne plasti so povezani izhodi vseh predhodnih plasti



Arhitektura ResNet (vir: [9])



Arhitektura DenseNet (vir: [9])

Konvolucijske nevronske mreže

(10)

Primer: Plasti mreže VGG in število prostih parametrov (vir: [4, 8])

```
INPUT: [224x224x3]      memory: 224*224*3=150K  weights: 0
CONV3-64: [224x224x64]  memory: 224*224*64=3.2M  weights: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64]  memory: 224*224*64=3.2M  weights: (3*3*64)*64 = 36,864
POOL2: [112x112x64]    memory: 112*112*64=800K  weights: 0
CONV3-128: [112x112x128] memory: 112*112*128=1.6M weights: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128] memory: 112*112*128=1.6M weights: (3*3*128)*128 = 147,456
POOL2: [56x56x128]     memory: 56*56*128=400K  weights: 0
CONV3-256: [56x56x256] memory: 56*56*256=800K  weights: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256] memory: 56*56*256=800K  weights: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256] memory: 56*56*256=800K  weights: (3*3*256)*256 = 589,824
POOL2: [28x28x256]     memory: 28*28*256=200K  weights: 0
CONV3-512: [28x28x512] memory: 28*28*512=400K  weights: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512] memory: 28*28*512=400K  weights: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512] memory: 28*28*512=400K  weights: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512]     memory: 14*14*512=100K  weights: 0
CONV3-512: [14x14x512] memory: 14*14*512=100K  weights: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K  weights: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K  weights: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512]       memory: 7*7*512=25K   weights: 0
FC: [1x1x4096]          memory: 4096  weights: 7*7*512*4096 = 102,760,448
FC: [1x1x4096]          memory: 4096  weights: 4096*4096 = 16,777,216
FC: [1x1x1000]          memory: 1000  weights: 4096*1000 = 4,096,000

TOTAL memory: 24M * 4 bytes ~= 93MB / image (only forward! ~*2 for bwd)
TOTAL params: 138M parameters
```

Inženirski pristop h globokim nevronske mrežam

A. Praktični razvojni proces pri delu z globokimi NN [1]:

1. Določiti cilje, metrike napake in ciljne vrednosti teh metrik
 - ▷ Metrike: izguba (ni popolnoma merodajna), točnost (*accuracy*), natančnost (*precision*), priklic (*recall*), krivulja PR, "F-score", pokritost (*coverage*)...
2. Čim prej vzpostaviti delujočo rešitev (*end-to-end pipeline*) in oceniti metrike zmogljivosti
3. Dobro preučiti sistem in določiti ozka grla v zmogljivosti
 - ▷ Npr. diagnosticirati, katere enote deluje slabše kot pričakovano; slabša zmogljivost zaradi pod- ali nadprileganja...
4. Ponavljajoče vnašati inkrementalne spremembe v sistem na osnovi izsledkov diagnosticiranja
 - ▷ Npr. zbirati nove učne podatke, uglasiti hiperparametre, spremeniti algoritme...

B. Predobdelava podatkov

- ▷ Tipično: ničelno povprečje, normalizacija (deljenje s standardnim odklonom)
- ▷ Redkeje (ne uporabljamo pri CNN!): PCA, redukcija s PCA (obdržimo značilnice z največ variance), beljenje
- ▷ **Pogosta napaka:** Statistike (npr. povprečje) izračunamo na celotni množici podatkov, uporabimo na vseh podmnožicah.
- ▷ **Pravilno:** Statistike izračunamo le na učni množici, uporabimo na vseh podmnožicah!

Inženirski pristop h globokim nevronske mrežam (2)

C. Inicializacija uteži

- ▷ **Pogosta napaka:** Uteži postavimo na 0.
 - ★ Posledica: Ni asimetrije med nevroni, vsi identično spreminjajo uteži med učenjem.
- ▷ Uteži postaviti na majhna naključna števila (npr. Gaussova porazdelitev $N(0, 1)$)
 - ★ Posledica: majhni gradienti med učenjem, majhna sprememba uteži (ni dobro!).
- ▷ Redka inicializacija
 - ★ Uteži vseh nevronov najprej na 0. Nato nevron naključno povežemo z nekaj nevroni na naslednji plasti (do 10). Uteži določimo po $N(0, 1)$.
- ▷ Tipično: Kalibrirati varianco z $1/\sqrt{n}$
 - ★ Vektor uteži nevrona po naključni inicializaciji množimo z $1/\sqrt{n}$, kjer n število vhodov v nevron.
 - ★ **Nevroni z ReLU aktivacijo:** množimo z $\sqrt{2/n}$

D. Inicializacija pragov

- ▷ Tipično: Prage postavimo na 0

Inženirski pristop h globokim nevronske mrežam (3)

E. Normalizacija množice (batch normalization)

- ▷ V globoke NN se uvaja kot posebna plast pred aktivacijsko plastjo (npr. pred ReLU).
- ▷ Faza učenja: Vsako značilnico normaliziramo kot

$$y_i = \gamma \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

- ★ parametra γ in β učimo
- ★ μ_B in σ_B povprečje in standardni odklon množice B , ϵ majhno število >0
- ▷ Faza "inference": Namesto plasti uporabimo zgornjo linearno enačbo, kjer γ in β fiksna, μ in σ določimo iz tekočega povprečenja preko vseh množic B .
- ▷ **V praksi: NN z normalizacijo množice so bolj robustne na slabo inicializacijo!**

F. Regularizacija

Kontrolira kapaciteto NN in preprečuje nadprileganje.

- ▷ L^2 regularizacija (Tikhonova regularizacija, razkroj uteži) – vse uteži imajo majhne vrednosti blizu 0
- ▷ L^1 regularizacija – vektor uteži redek in blizu 0, določene uteži postanejo 0

Inženirski pristop h globokim nevronske mrežam (4)

- ▷ "Dropout"
 - ★ Zgolj v fazi učenja!
 - ★ Med učenjem obdržimo nevron aktiven z verjetnostjo p (tipično $p = 0,5$)
 - ★ Kombiniramo z drugimi regularizacijskimi tehnikami.
- ▷ Umetno povečanje podatkovne množice (*dataset augmentation*)
 - ★ Ideja: Model strojnega učenja generalizira bolje, če ga učimo na več podatkih.
 - ★ Postopek: S pomočjo različnih tehnik in transformacij generiramo nove lažne podatke in jih dodamo v učno množico.
 - ★ Transformacije: Rotacija, skaliranje, sprememba svetlosti, zamegljenje...
- ▷ Zgodnja prekinitev (*early stopping*)
 - ★ Med učenjem opazujemo napako na validacijski množici.
 - ★ Če se napaka po p korakov ne zmanjša, prekinemo učenje. Vrnemo parametre (in število učnih korakov), s katerimi dobili najmanjšo napako.
 - ★ Potrebujemo učno in validacijsko množico! Slabost: ne izrabimo vseh podatkov za učenje!
 - ★ Če po zgodnji prekinitvi uporabimo vse podatke (učna + validacijska množica):
 1. učimo od začetka do optimalnega števila korakov
 2. nadaljujemo učenje z optimalnimi parametri

Inženirski pristop h globokim nevronske mrežam (5)

G. Izbira hiperparametrov

Ročno ugaševanje hiperparametrov

- Cilj: poiskati takšne hiperparametre, da dobimo najmanjšo napako generalizacije glede na sprejemljiv "strošek" računanja in pomnilnika.

Hiperparameter	Kapaciteta zviša, če	Opombe
Št. skritih enot	Povečamo	Povečamo kapaciteto, a tudi strošek vsake operacije v modelu
Stopnja učenja	Uglašena optimalno	Nepravilna nastavitev pomeni neuspeh optimizacije
Velikost konv. jeder	Povečamo	Več parametrov, a izhod manjše dimenzije (se lahko zmanjša tudi kapaciteta)
Ničelna obroba	Povečamo	Ohranja velikost reprezentacije modela
Koefic. razkroja uteži (L^2)	Zmanjšamo	Sprosti modelne parametre, da postanejo večji
Stopnja "dropout"	Zmanjšamo	Manj pogosto izločanje enote, večja možnost prilaganja učni množici

Inženirski pristop h globokim nevronske mrežam (6)

Avtomatsko izbiranje hiperparametrov

- ▷ Avtomatski algoritmi za optimizacijo hiperparametrov
 - ★ Npr. evolucijski in drugi algoritmi po vzoru iz narave
- ▷ Iskanje po mreži (*grid search*)
 - ★ Diskretiziramo hiperparametre, izvedemo učenje z eno kombinacijo hiperparametrov, ocenimo napako na validacijski množici. "Optimalni" nabor, kjer napaka najmanjša.
 - ★ Smiselno: grobo-fini pristop (v okolici "optimuma" bolj fino razdelimo območje).
- ▷ Naključno iskanje (*random search*)
 - ★ Hiperparameter ne diskretiziramo, ampak ga izbiramo po določeni porazdelitvi.
 - ★ Smiselno: grobo-fini pristop
 - ★ V splošnem učinkovitejši od iskanja po mreži.

Inženirski pristop h globokim nevronske mrežam (7)

H. Privzeti izhodiščni modeli (default baseline models)

- ▷ Ideja: Čim prej vzpostaviti delujočo rešitev (*end-to-end pipeline*) in oceniti metrike zmogljivosti (točka A. Praktični razvojni proces...).
 - ★ **Metrike izhodiščnega modela služijo kot "benchmark" (jih želimo izboljšati!).**
- ▷ Pristop izberemo glede na kompleksnost reševanega problema:
 - ★ "AI-complete": Globoko učenje
 - ▷ Npr. prepoznavanje objektov, prepoznavanje govora, strojno prevajanje...
 - ★ Sicer: Preprostejše metode (npr. regresija)

Nekaj predlogov:

- ▷ Zvezni perceptroni – če vhod vektorji fiksne velikosti
- ▷ Konvolucijske NN – če vhod z znano topološko strukturo (npr. slike)
- ▷ Aktivacijska funkcija – ReLU ali njene posplošitve (npr. Leaky ReLU)
- ▷ Optimizacija – stohastični gradientni sestop (moment + padajoča stopnja učenja)
- ▷ Blaga regularizacija – Zgodnja prekinitev (vedno), "dropout" (smiselno), normalizacija množice (ne v začetne izhodiščne modele)
- ▷ Nadzorovano učenje (za večino problemov)
- ▷ Če že rešitev za soroden problem, uporabimo najboljši model in algoritme od tam

Inženirski pristop h globokim nevronske mrežam (8)

I. Spremljanje uspešnosti učenja, razhroščevanje in drugi napotki

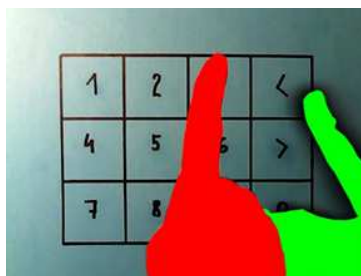
Preveriti oz. opazovati:

- ▷ Gradiente v mreži (uporablja se redkeje)
- ▷ Pričakovano izgubo pri naključnem delovanju (brez regularizacije; predpostavimo, da vse deluje napačno)
- ▷ Nadprilegamo zelo majhno množico podatkov (brez regularizacije) – strošek mora biti 0
- ▷ Izgubo skozi čas (epohe) v odvisnosti od stopnje učenja – mora počasi padati
 - ★ Spremenimo stopnjo učenja, velikost "mini-batch"
- ▷ Natančnost na učni in validacijski množici skozi čas (epohe) – razlika med natančnostima kaže velikost nadprileganja
 - ★ Velika razlika: večja regularizacija, povečamo množico podatkov
 - ★ Majhna razlika: povečamo kapaciteto modela
- ▷ Razmerje norma spremembe uteži proti normi uteži – želimo okrog 10^{-3}
 - ★ Sicer sprememba stopnje učenja
- ▷ Porazdelitev aktivacij / gradientov za posamezno plast – čudne porazdelitve kažejo problem napačne inicializacije
- ▷ Vizualizacijo uteži prve plasti CNN – uteži morajo biti lepe, gladke, čiste, različne
- ▷ Vizualizacijo modela v akciji – rezultati smiselni?
 - ★ Vizualizirati tudi rezultate, kjer se model najbolj moti

Aplikacije

Aplikacije konvolucijskih nevronske mrež

1. Segmentacija slik

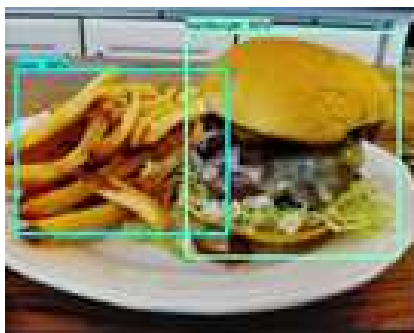


Vir: [10]

2. Preiskovanje slikovnih podatkovnih zbirk (*image retrieval*)

3. Razvrščanje slik

4. Razvrščanje in detekcija oz. lokalizacija



Vir: [11]



Sedenje z verjetnostjo 74 % (vir: [12])

5. Luščenje značilnic

Dodatna literatura

- [1] I. Goodfellow, Y. Bengio, A. Courville (2016), Deep learning, The MIT Press, London, VB.
- [2] Abto Software (2018), Kitchen Furniture & Appliances Recognition,
<https://www.abtosoftware.com/blog/kitchen-furniture-appliances-recognition-in-photos>
- [3] U. Karn (2016), An Intuitive Explanation of Convolutional Neural Networks,
<https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/>
- [4] A. Karpathy, J. Johnson (2018), CS231n Convolutional neural networks for visual recognition,
<http://cs231n.github.io/>
- [5] Y. Zhu (2015), All you want to know about CNNs...,
<http://www.cs.utoronto.ca/~fidler/teaching/2015/slides/CSC2523/CNN-tutorial.pdf>
- [6] F.-F. Li, J. Johnson, S. Yeung (2018), Lecture 5: Convolutional neural networks,
http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf
- [7] A. Deshpande (2016), The 9 Deep Learning Papers You Need To Know About (Understanding CNNs Part 3),
<https://adeshpande3.github.io/adeshpande3.github.io/The-9-Deep-Learning-Papers-You-Need-To-Know-About.html>
- [8] Visual geometry group (2014), Very Deep Convolutional Networks for Large-Scale Visual Recognition,
http://www.robots.ox.ac.uk/~vgg/research/very_deep/

- [9] J. Jordan (2018), Common architectures in convolutional neural networks, <https://www.jeremyjordan.me/convnet-architectures/>
- [10] A. Javornik (2017), Avtomatsko nastavljanje parametrov segmentacijske metode aplikacije virtualna tipkovnica s pomočjo nevronske mreže, diplomsko delo, UM-FERI, Maribor.
- [11] J. Banko (2018), Prepoznavanje jedi iz digitalnih slik s pomočjo konvolucijskih nevronskih mrež, diplomsko delo, UM-FERI, Maribor.
- [12] M. Baketarić (2018), Prepoznavanje aktivnosti osebe iz zaporedja slik s pomočjo konvolucijskih nevronskih mrež, diplomsko delo, UM-FERI, Maribor.

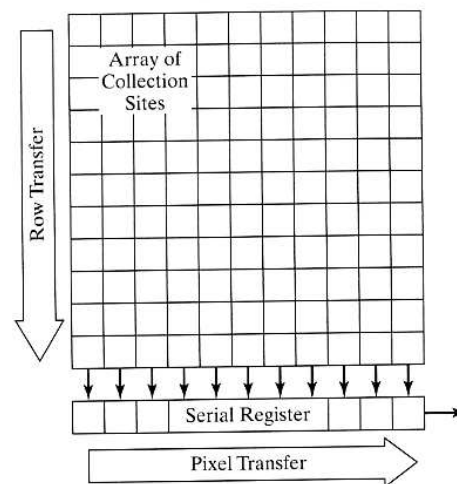
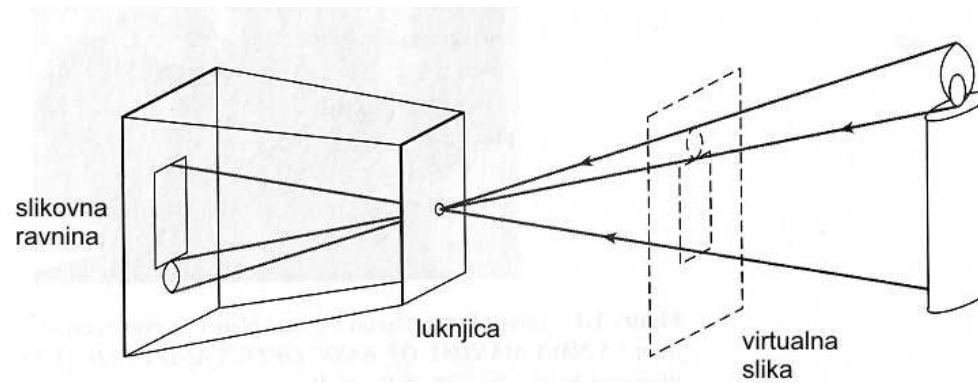
2. Napredni koncepti zajemanja slik

Hiperspektralno zajemanje

Polarizirano zajemanje

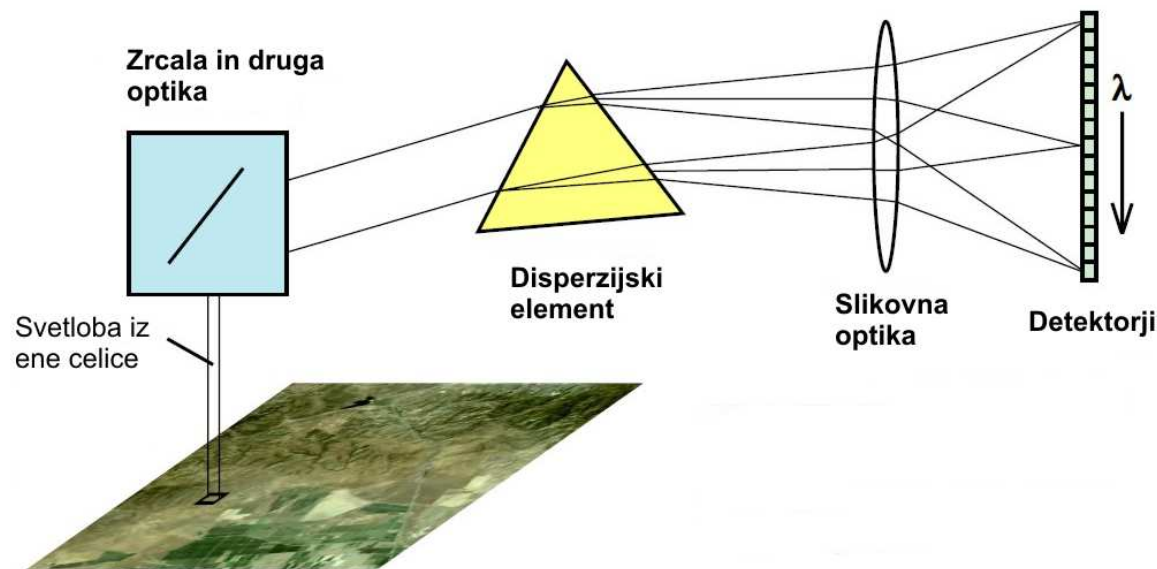
Interferometrija

Zajemanje slik s kamerami CCD



Hiperspektralno zajemanje

- ▷ Hiperspektralne slike (*Hyperspectral images*) lahko dobimo s pomočjo kompleksnih senzorjev slikovnih spektrometrov (*imaging spectrometers*)
- ▷ Razvoj teh senzorjev omogočila združitev dveh tehnologij:
 - ★ spektroskopija (*spectroscopy*): študij od materialov (objektov) oddane oz. odbite svetlobe in spremljanje sprememb v energiji v odvisnosti od valovne dolžine.
 - ★ daljinsko slikanje (*remote imaging*) Zemlje in drugih planetov: takšni senzorji zmožni fokusirati in izmeriti odbito svetlobo iz mnogih, blizu ležečih področij na Zemljini površini.

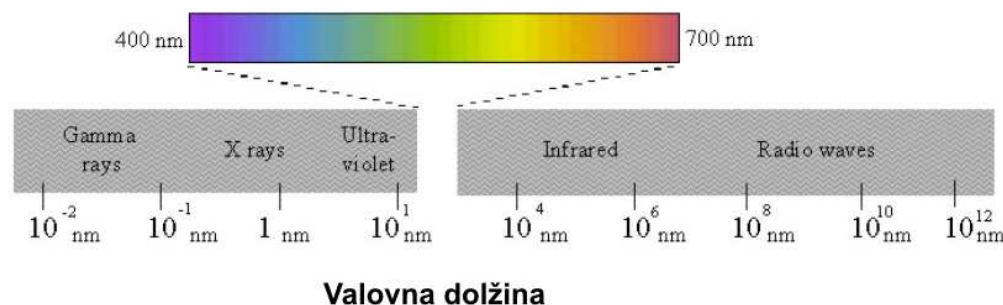


Shematski diagram osnovnih elementov slikovnega spektrometra (vir: [1]).

Hiperspektralno zajemanje

(2)

- ▷ Elektromagnetno sevanje (npr. svetlobo) opredelimo z valovno dolžino (vir: [2]).

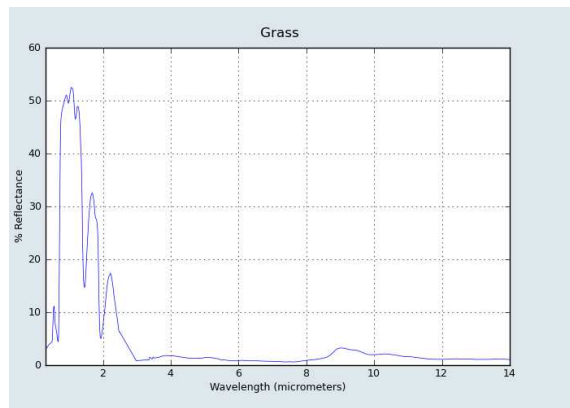


Razlika med multispektralnim in hiperspektralnim zajemanjem

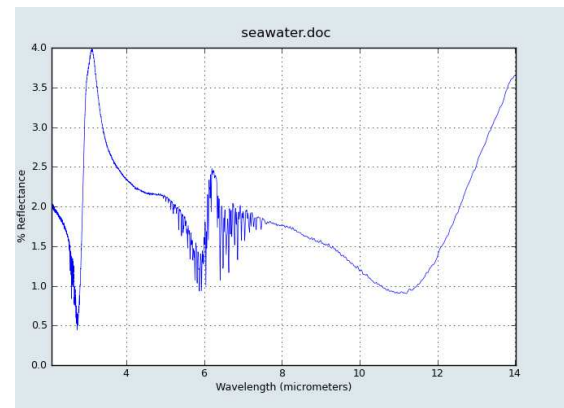
- ▷ Multispektralno zajemanje:
 - ★ merjenje odbitega sevanja s površin zgolj pri nekaj razmaknjenih valovnih dolžinah, pri čemer je opazovan pas valovnih dolžin širok.
- ▷ Hiperspektralno zajemanje:
 - ★ merjenje odbitega sevanja pri množici sosednjih valovnih dolžin, pri čemer je opazovan pas valovnih dolžin ozek.
- ▷ Ni zgolj število pasov valovnih dolžin ključno za hiperspektralnost, ampak predvsem ozko opazovano območje valovnih dolžin (majhen razmik med sosednjimi pasovi)!
- ▷ **Primer:**
 - ★ hiperspektralno – 20 valovnih dolžin, ki so 10 nm narazen
 - ★ multispektralno – 20 valovnih dolžin, ki so 100 nm narazen

Kaj pridobimo s takšnim zajemanjem?

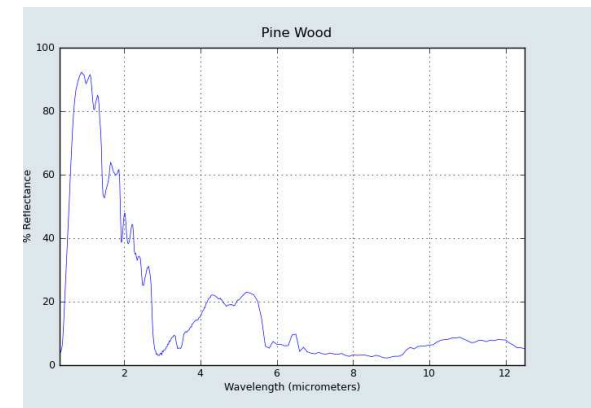
- ▷ Spekter koeficienta odbojnosti (*reflectance*): prikazuje koeficient odbojnosti materiala, izmerjen v določenem pasu valovnih dolžin (analogija velja pri koeficientu absorpcije)
 - ★ **Koeficient odbojnosti:** odstotek svetlobe, ki je dosegla material in se od njega odbila
- ▷ Spekter koeficienta odbojnosti in absorpcije, opazovan preko množice valovnih dolžin, lahko enolično identificira opazovane materiale.
- ▷ **Primer:**



Trava



Morje

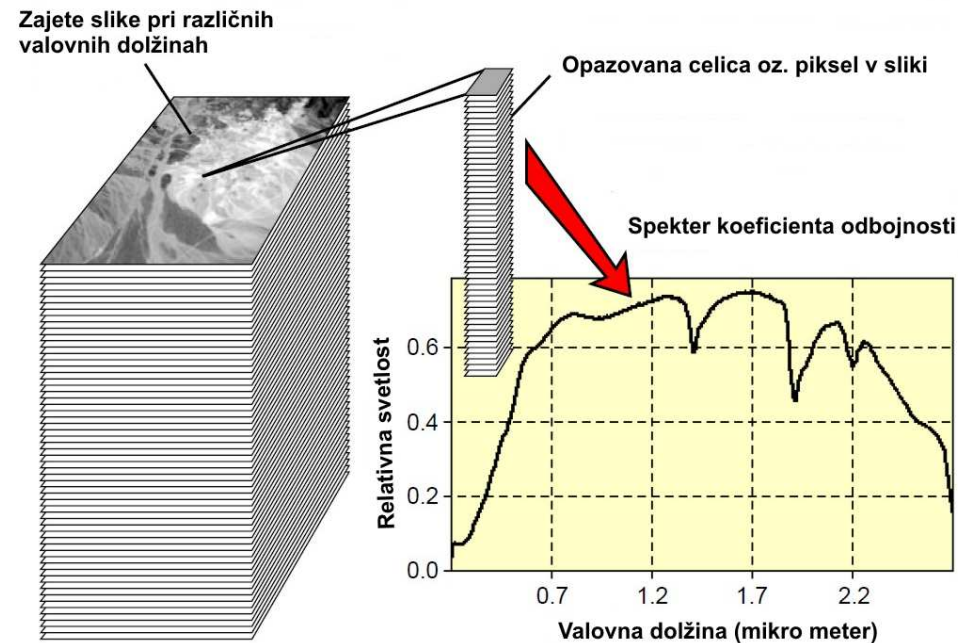


Borovec

Hiperspektralno zajemanje

(4)

- ▷ Slikovni spektrometer omogoča, da dobimo zvezni spekter koeficienta odbojnosti za vsako slikovno točko.



Vir: [1]

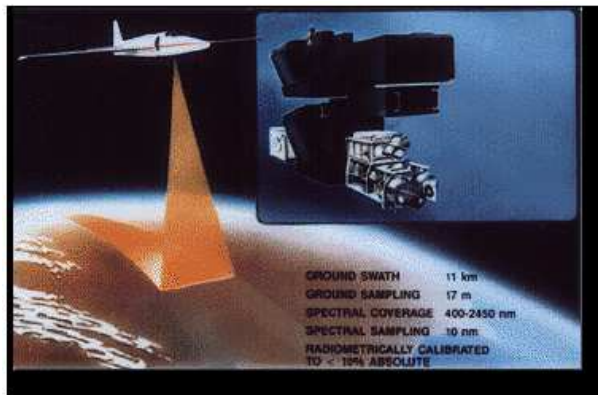
- ▷ Obstajajo knjižnice s spektri koeficientov odbojnosti za naravne in umetno narejene materiale:
 - ★ Knjižnica ASTER (<http://speclib.jpl.nasa.gov>): vsebuje čez 2400 spektrov, pri čemer je pokrit pas valovnih dolžin med 0,4 in 14 μm
 - ★ Knjižnica USGS (<http://speclab.cr.usgs.gov/spectral-lib.html>): vsebuje čez 1300 spektrov, pri čemer je pokrit pas valovnih dolžin med 0,2 in 3 μm

Hiperspektralno zajemanje

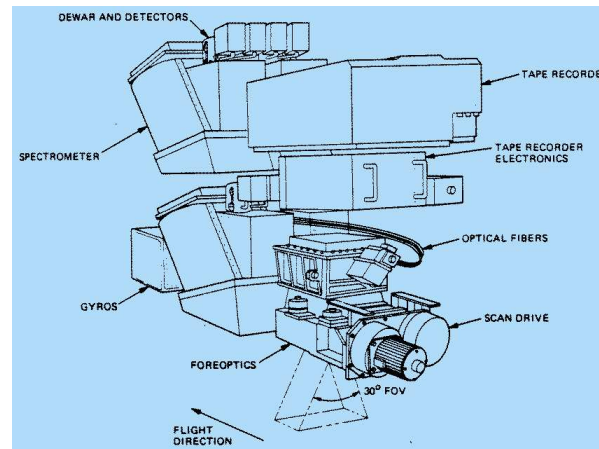
(5)

- ▷ Slikovni hiperspektralni spektrometri lahko nameščeni na:
 - ★ letečo napravo (*airborne sensors*): npr. AVIRIS, AISA, DAIS 2115, ...
 - ★ na satelit (*space-based sensors*): npr. PRISM, PROBA, NEMO, ...

Primer: AVIRIS (Airborne Visible Infrared Imaging Spectrometer),
<http://aviris.jpl.nasa.gov>



Zajem



Inštrumenti

Detajli:

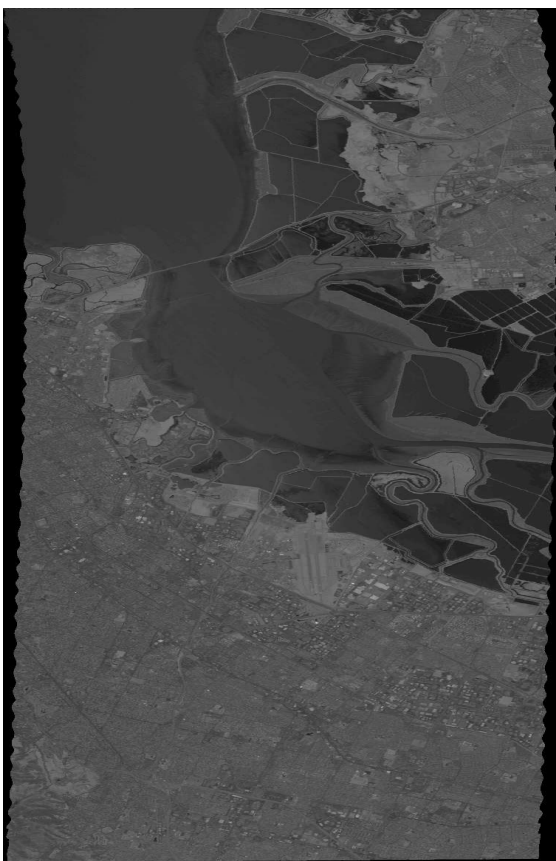
- ▷ Višina zajema: 20 km nad Zemljo (ER-2 jet)
- ▷ Število pasov oz. valovnih dolžin: 224 pasov, razmik med pasovi 10 nm
- ▷ Spektralni obseg: 0,4 do 2,5 μm
- ▷ Snemalni pas (*ground swath*): 11 km
- ▷ Ločljivost: 1 piksel ustreza kvadratu 20 × 20 m na Zemlji

Hiperspektralno zajemanje

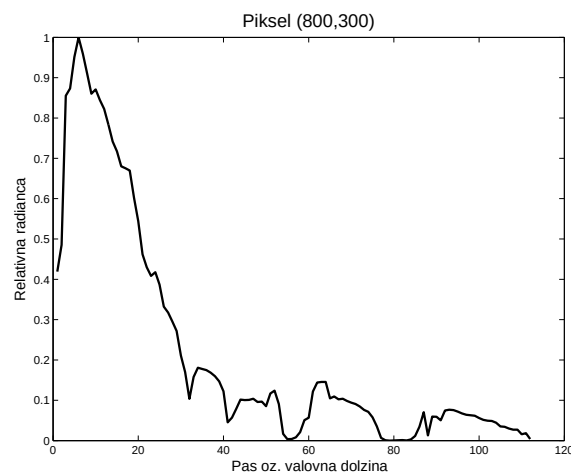
(6)

Letališče Moffett (USA):

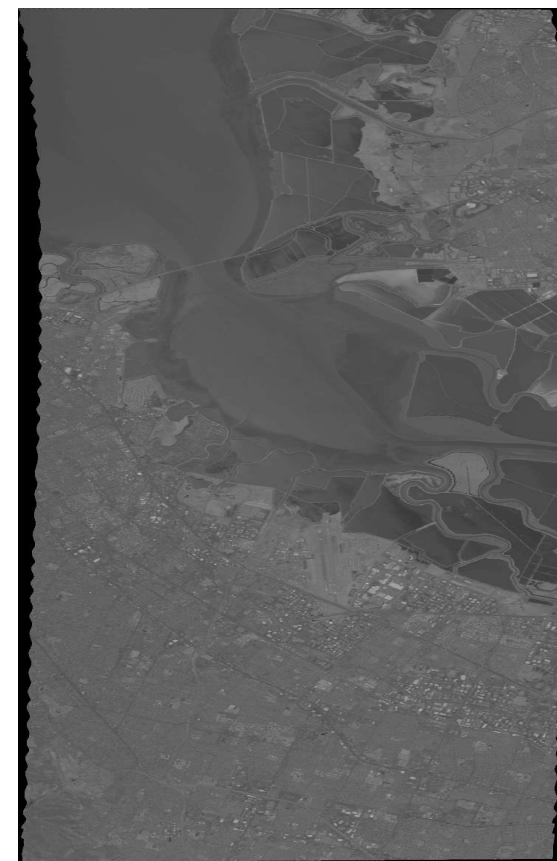
- ▷ Slika velikosti: 1924×753 pikslov $\times$ 224 pasov ($\approx$ 618 MB)



Pas 140



Profil za piksel (800,300)



Povprečje skozi vse pasove

Analiza hiperspektralnih slik

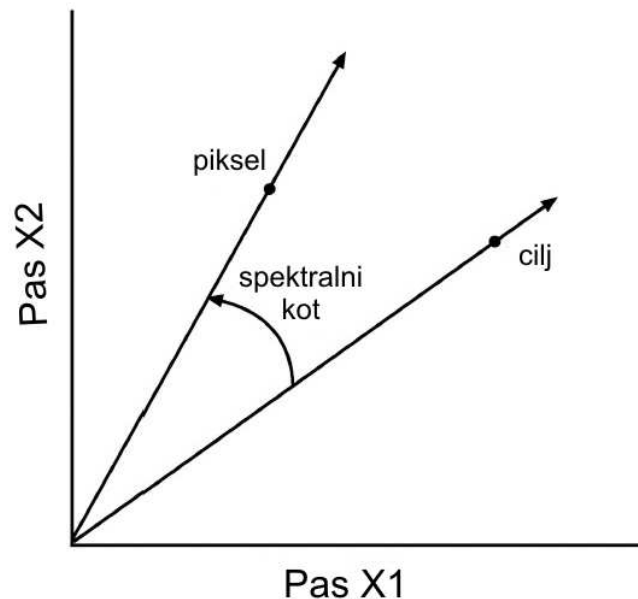
- ▷ Tipično: primerjamo spekter izbranega piksla z referenčnim spektrom (t.i. ciljni spekter)
 - ★ ciljni (*target*) spekter lahko dobimo iz knjižnic s spektri odbojnih koeficientov
- ▷ Metode analiziranja:
 - ★ Metode celega piksla
 - ★ Podpikselne metode (*sub-pixel methods*)

A. Metode celega piksla

- ▷ Skušajo določiti, če je množica ciljnih materialov prisotnih v izbranem pikslu
 - ★ izračun na osnovi spektralne podobnosti med pikslom in ciljnim spektrom(tri)
 - ★ metode: najmanjša razdalja, največja verjetnost, preslikava spektralnega kota, prileganje spektralnih značilk...

Preslikava spektralnega kota (*spectral angle mapper*)

- ▷ Spekter piksla je točka v n -dimenzionalnem prostoru (vsaka valovna dolžina ena dimenzija)
- ▷ **Primer:** $n = 2$



- ▷ Spektralni kot je relativno neodvisen na spremembe v svetlosti
 - ★ to vpliva le na velikost vektorja, ne pa na smer!
- ▷ Treba je izračunati spektralni kot med vsakim pikslom in vsakim ciljem!

B. Podpikselske metode

- ▷ Gre za izračun deleža prisotnosti ciljnih materialov v vsakem pikslu slike (en piksel lahko vsebuje več različnih materialov).
- ▷ S takšno analizo lahko detektiramo množino ciljev, ki so veliko manjši kot pa je velikost piksla.

Popolno linearno spektralno razmešanje (*complete linear spectral unmixing*)

- ▷ V sceni poznamo množico spektralno edinstvenih (čistih) materialov (t.i. spektralni končni člani (*endmembers*) za to sceno).
- ▷ Linearno spektralno razmešanje temelji na ideji:
 - ★ utež vsakega končnega člana je direktno proporcionalna ploščini znotraj piksla, ki ga le-ta pokriva.
- ▷ Postopek razmešanja:
 1. Spektri vseh končnih članov v sceni morajo biti znani.
 2. Reši sistem n linearnih enačb za vsak piksel (n – število končnih članov, e_i – i -ti končni član):

$$\varsigma(\mathbf{p}) = \alpha_1 \varsigma(e_1) + \alpha_2 \varsigma(e_2) + \dots + \alpha_n \varsigma(e_n)$$

- ▷ Za enolično rešitev potrebujemo vsaj n enačb, kar ni problematično pri hiperspektralnih slikah.

- ▷ Rezultat:
 - ★ dobimo sliko vsebnosti (*abundance image*) za vsakega končnega člana
 - ★ iz te slike odčitamo, kolikšen delež končnega člana je vsebovan v izbranem pikslu

Primer: Slike vsebnosti (vir: [1])



Delež vegetacije



Delež vode



Delež tal

Delno razmešanje

- ▷ detektira prisotnost in količino vsebnosti enega samega ciljnega materiala

Hiperspektralno zajemanje

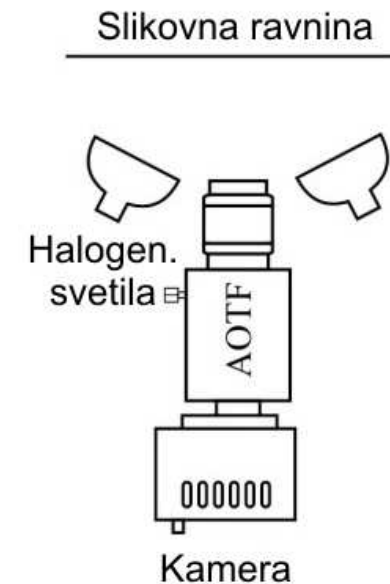
(11)

Hiperspektralne slike ne najdemo samo pri daljinskem snemanju!

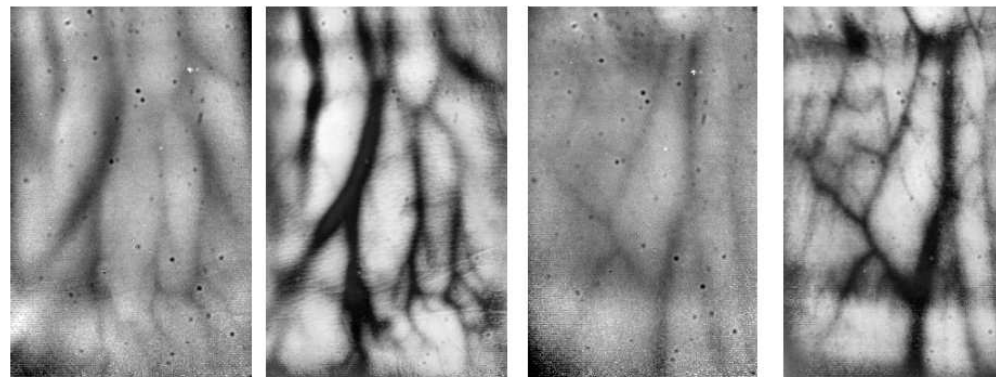
Primer: Izboljšanje kontrasta slik podkožnih ven [3]

▷ Uporabljeni dve prikazani enoti:

1. Kamera CCD Sony XC55N – pokriva spekter med 550 in 1000 nm
 2. Kamera InGaAs, občutljiva na bližnjo IR svetlobo – pokriva spekter med 900 in 1700 nm
- ★ razmik med pasovi 10 nm
 - ★ frekvenca zajema: 100 fps
 - ★ 115 slik ob zajemu: 34 iz prve enote in 81 iz druge enote



AOTF – acousto-optical tunable filters



a) originalna slika – zapestje, b) izboljšana slika, c) originalna slika – komolec in d) izboljšana slika

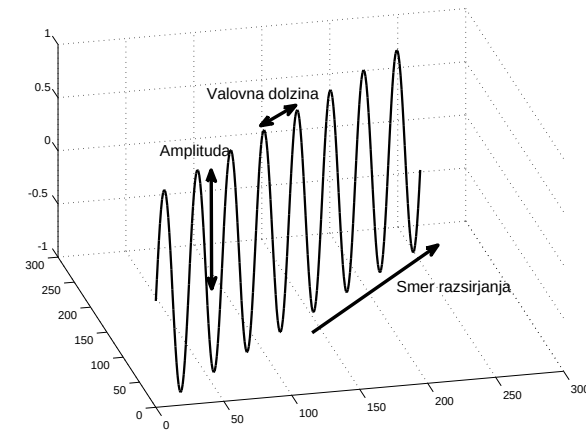
Aplikacije na osnovi analize hiperspektralnih slik

- ▷ geologija:
 - ★ kartografiranje mineralov
 - ★ detektiranje lastnosti tal (npr. ocenjevanje vlažnosti, slanosti, prisotnosti organskih snovi)
- ▷ vegetacija
 - ★ identificiranje vrst vegetacije
- ▷ vojska:
 - ★ detektiranje vojaških vozil, skritih pod vegetacijo

Polarizirano zajemanje

Osnovni pojmi

- ▷ Polarizacija je splošen deskriptor svetlobe in vsebuje informacijo o objektih, ki odbijajo svetlobo:
 - ★ Tradicionalni senzorji na osnovi intenzitete to informacijo ignorirajo!
- ▷ Trije neodvisni pojmi, s katerimi opišemo svetlobo:
 - ★ valovna dolžina – določa barvo
 - ★ amplituda – določa svetlost oz. intenziteto
 - ★ smer vibriranja – polarizacija
- ▷ Iz polarizacije lahko določimo podrobnosti o izvoru svetlobe in površini, ki svetlobo odbija.
- ▷ Človeško oko ni zmožno vizualizirati polarizacije svetlobe, kar je za živali pogost pojav:
 - ★ čebele – polarizacijo uporabljajo kot kompas
 - ★ določeni insekti (npr. vodni pajek) in kačji pastir – za detektiranje vode in podvodnih predatorjev
 - ★ ligenj – lahko modulira odbojnost svoje kože (za kamuflažo)



Polarizirano zajemanje

(2)

Podrobneje o polarizaciji

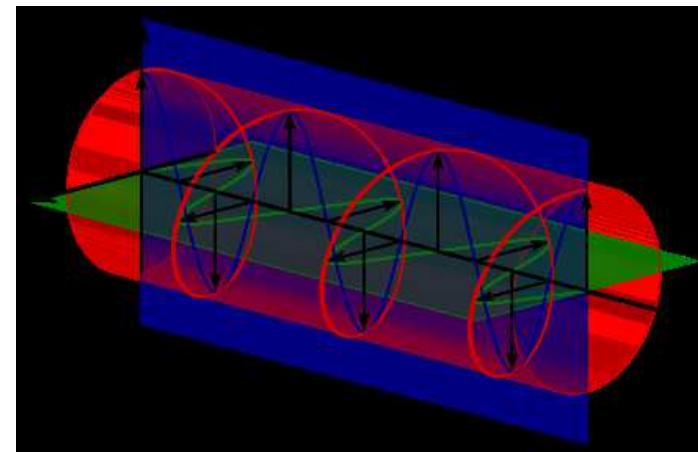
- ▷ **Tipi polarizacije** – polarizacija se nanaša na orientacijo električnega polja elektromagnetnega valovanja

- ★ **linearna polarizacija:**

- ▷ vektor električnega polja oscilira zgolj v eni ravnini
- ▷ v naravi: pri odboju svetlobe od površine (npr. odboj od jezera)

- ★ **krožna polarizacija:**

- ▷ kombinacija dveh linearnih valovanj, pravokotnih ena na drugo, ki sta za 90° iz faze
- ▷ vektor električnega polja opisuje krožnico skozi čas



Krožna polarizacija

- ▷ Polarizacijo svetlobnega valovanja popolnoma popišemo s **Stokesovim vektorjem**, ki sestoji iz 4 komponent:

- ★ s_0 – skupna intenziteta svetlobe
- ★ s_1 – popisuje horizontalno in vertikalno polarizacijo
- ★ s_2 – popisuje komponente pri $\pm 45^\circ$
- ★ s_3 – popisuje desno in levo krožno polarizacijo

Slikovni polarimetri (*imaging polarimeters*)

- ▷ So naprave, ki izmerijo Stokesov vektor za vsako točko v sliki
- ▷ Najdemo jih v različnih izvedbah
 - ★ Časovno sekvenčni (*time sequential*) – spreminjajo konfiguracijo svojih inštrumentov med meritvami
 - ★ Na osnovi delitve amplitude – npr. uporaba "beamsplitter"-ja
 - ★ Na osnovi delitve luknjice (fronte valovanja)

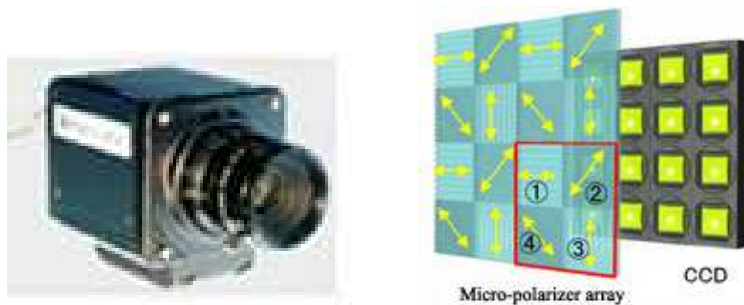
Polarimeter na osnovi delitve fronte valovanja

- ▷ Fronta valovanja (*wavefront*) je množica točk z isto fazo.
 - ★ Pri razširjanju valovanja v 3D je to ploskev.
- ▷ Ta vrsta polarimetrov deluje na podobni osnovi kot kamere CCD, ki zajemajo barvne slike s pomočjo Bayerjevega filtra.
- ▷ Koncept zajemanja:
 1. Vsak piksel zajema določeno polarizacijsko stanje, ki se razlikuje od stanja sosed.
 2. S pomočjo interpolacije in registracije ocenimo polarizacijo v vsakem pikslu.
- ▷ Robustno zajemanje, omejeni smo le s hitrostjo kamere in s postopkom interpolacije.

Polarizirano zajemanje

(4)

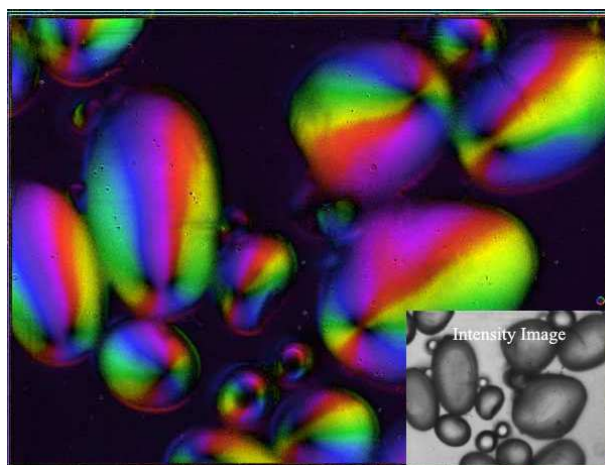
Primer: Polarizacijska slikovna kamera PI-100 [4]



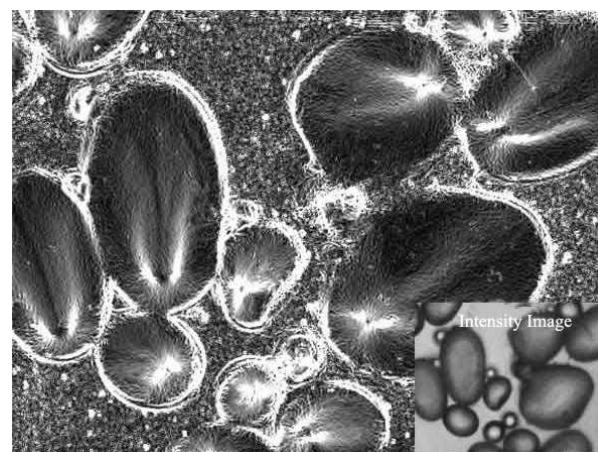
▷ Tehnične podrobnosti:

- ★ velikost delovnega območja 1024×868 pikslov, hitrost zajema 8 fps, spektralno območje 500–550 nm

▷ Primeri slik:



Polarizacija krompirjevega škroba pod mikroskopom.



Slika diferencialnega polarizacijskega azimuta škrobnega zrna. Vidna je interna struktura.

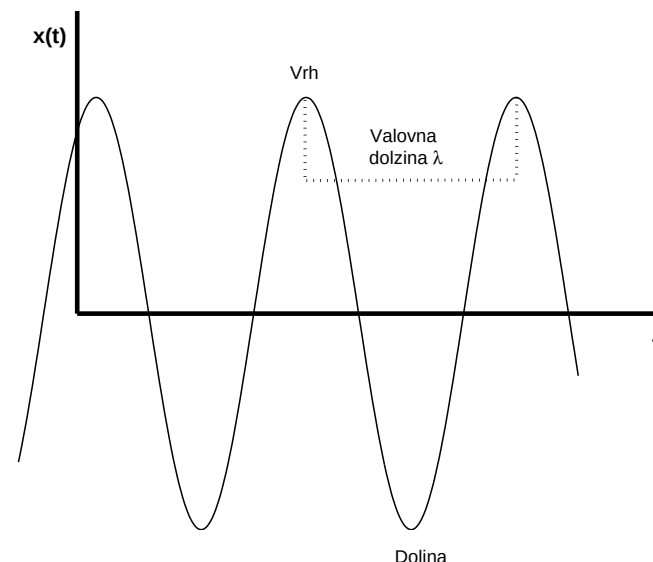
Aplikacije

- ▷ Identificiranje defektov v proizvodnji letal.
- ▷ "Pogled v vodo":
 - ★ npr. detektiranje oljnih madežev
- ▷ Avtomatsko detektiranje umetnih objektov v naravnih okoljih.
- ▷ Odstranjevanje bleščanja iz slik.

Interferometrija

Nujni pojmi za razumevanje interferometrije

- ▷ Svetlobni vir oddaja svetlobno valovanje.
- ▷ Valovi se nenehno razširjajo iz svetlobnega vira v vse smeri
 - ★ podobno kot valovanje na jezeru, ko vržemo vanj kamen
- ▷ **Bela svetloba** (npr. sonce) – vsebuje vse valovne dolžine, na katere je človeško oko občutljivo
- ▷ Za interferometrijo je najbolje uporabiti svetlobo ene same valovne dolžine (t.i. monokromatsko svetlobo)
 - ★ svetlobni vir mora biti še **koherenten** (*coherent*) – vsi svetlobni valovi potujejo v fazi
 - ★ najbolj primerna je laserska svetloba



Kaj je interferenca?

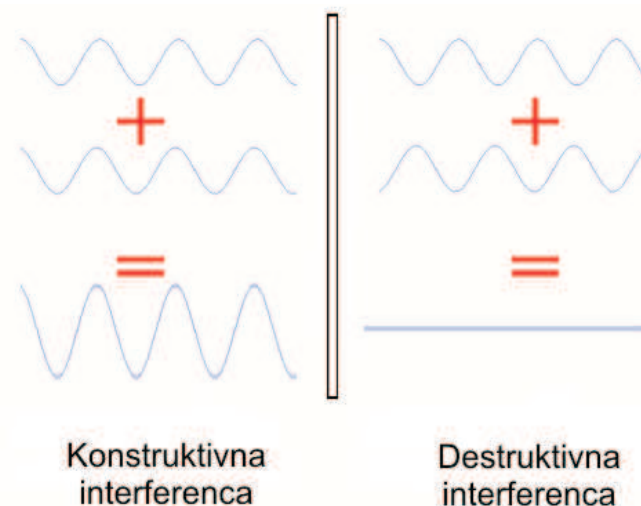
- ▷ Interferenca se pojavi, ko se srečata in prekrijeta dve valovanji, ki nosita energijo:
 - ★ namesto dveh valovanj dobimo eno novo
 - ★ oblika in velikost novega valovanja je odvisna od originalnih valovanj
 - ★ takšnemu združevanju valovanja rečemo tudi **superpozicija** (*superposition*)

- ▷ **Konstruktivna interferenca**

- ★ dobimo valovanje z isto valovno dolžino in frekvenco
- ★ amplituda je večja (višji vrhovi)

- ▷ **Destruktivna interferenca**

- ★ vrhovi v enem valovanju se izničijo z dolinami v drugem valovanju



Dve ekstremni situaciji:

- ▷ Konstruktivna interferenca – dve identični valovanji sta v fazi
 - ★ dobimo novo valovanje identično osnovnemu, le vrhovi so $2\times$ večji
- ▷ Destruktivna interferenca – dve identični valovanji sta popolnoma iz faze (sta v antifazi)
 - ★ kot rezultat dobimo nič
- ▷ Množica možnosti med obema ekstremoma!

Interferometrija

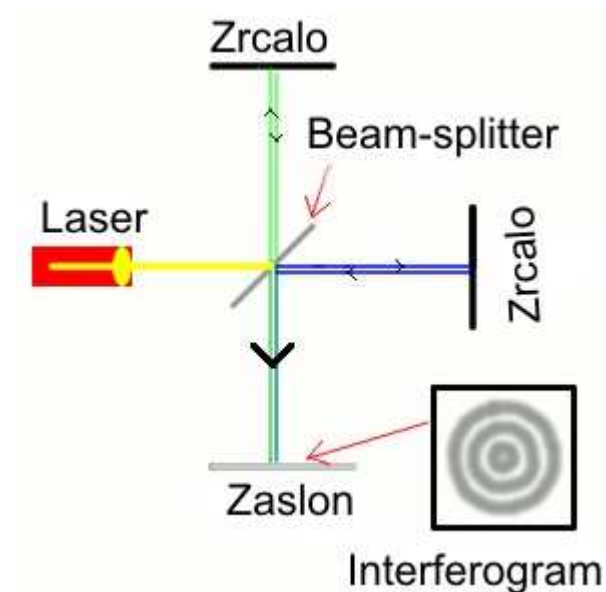
(3)

Interferometer

- ▷ Naprava (oz. sistem), ki je zmožna z združevanjem svetlobe različnih virov ustvariti interferogram in ga vizualizirati (oz. posneti s kamero CCD).
- ▷ Interferometer je izredno natančen znanstveni instrument, namenjen merjenju z izredno natančnostjo.

Kako deluje? Razlaga za dva žarka (oz. valovanja).

- ▷ Uporabimo svetlobni vir (tipično laser) in ga razdelimo v dva žarka.
 - ★ "Beam-splitter" – polprosojno steklo ali pol steklo (tj. steklo narahlo obloženo s srebrom)
- ▷ Referenčni žarek
 - ★ potuje do zrcala in se odbije do zaslona (kamere CCD)
- ▷ Testni žarek
 - ★ sveti na oz. skozi merjenec (objekt), do drugega zrcala, se odbije ter gre skozi "beam-splitter" do zaslona

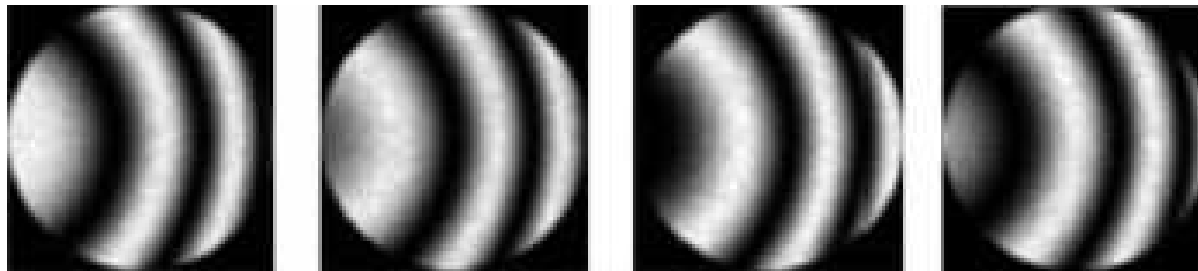


Michelsonov interferometer

- ▷ Testni žarek potuje malenkost daljšo pot (oz. po drugačni poti) kot prvi žarek.
 - ★ Žarka sta iz faze.
 - ★ Posledica: pride do interference med žarkoma.

Interferogram

- ▷ Rezultat interference med žarkoma.
- ▷ Nastane vzorec svetlih in temnih področij, ki ga lahko vizualiziramo.

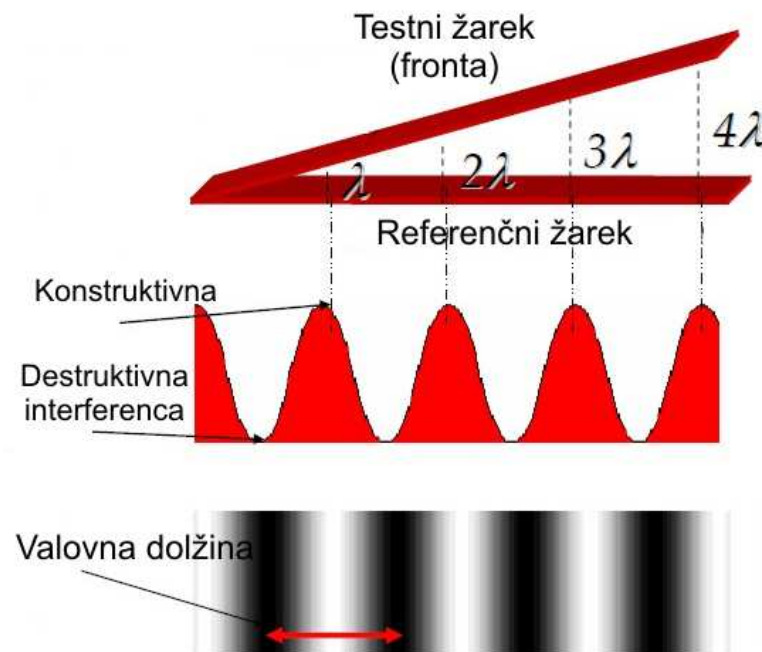


- ▷ Interferogram vsebuje pasove oz. **interferenčne resice** (*interference fringes*)
 - ★ temna področja – področja, kjer je prišlo do destruktivne interference
 - ★ svetla področja – področja, kjer je prišlo do konstruktivne interference
- ▷ Oblika vzorca interference odvisna od:
 - ★ različnosti poti med žarkoma in/ali
 - ★ dodatne razdalje, ki jo je prepotoval eden od žarkov.
- ▷ Z opazovanjem in merjenjem resic lahko oba pojava (različnost poti, dodatna razdalja) izračunamo z veliko natančnostjo
 - ★ interferometri lahko teoretično merijo razdalje z natančnostjo reda μm

Oblike interferograma

A. Žarka z valovno fronto v ravnini (*flat wavefront*)

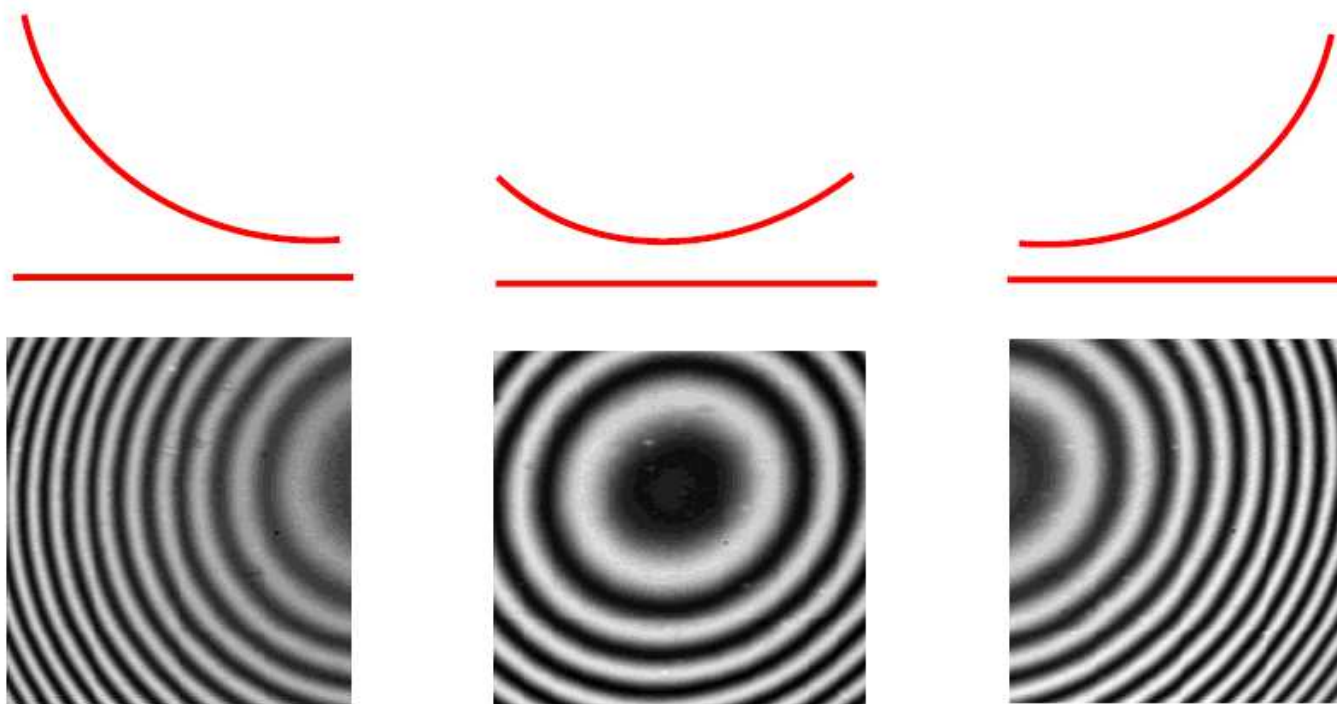
- ▷ Če sta žarka popolnoma pravokotna oz. je en žarek rahlo nagnjen
 - ★ rezultat: Interferogram z ravnimi, paralelnimi svetlo-temnimi pasovi



- ▷ Razmik med dvema resicama iste barve je velikosti 1 valovne dolžine.
- ▷ Z nagibanjem enega od obeh žarkov dobimo več resic:
 - ★ velikost nagiba določimo kot število pasov v luknjici (zaslonki).

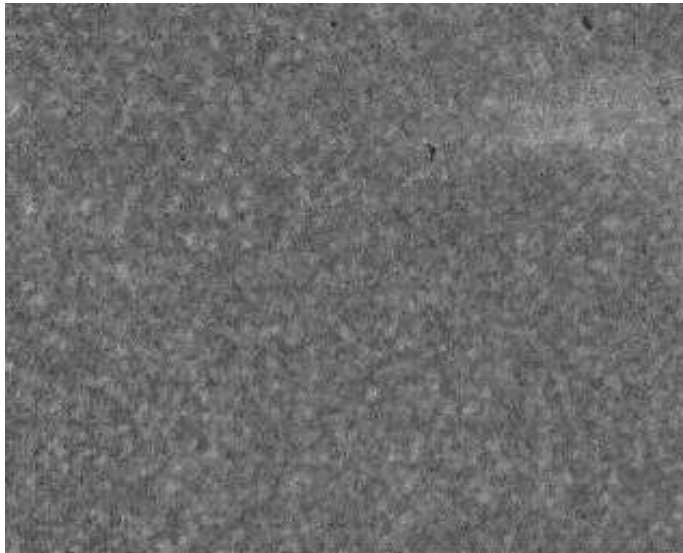
B. Žarek s sferično valovno fronto, drugi pa s fronto v ravnini

- ▷ Rezultat: Interferogram z ukrivljenimi resicami
- ▷ Če ni nagiba, potem so resice popolnoma krožne.



Kje še najdemo interferometrijo?

- ▷ Radarska interferometrija
- ▷ Mikroskopi na osnovi interferometrije
- ▷ Prototipna aplikacija (Laboratorija SPO in LEOSS): Spremljanje vitalnih življenjskih znakov s pomočjo optičnega vlakna in kamere
 - ★ Določanje trenutkov dihanja in srčnih utripov
 - ★ Interferenca na osnovi več svetlobnih izvorov (večrodovno optično vlakno)
 - ★ Gre za t.i. **pegasto interferometrijo**



Dodatna literatura

- [1] R.B. Smith (2006), Introduction to hyperspectral imaging, MicroImages inc., <http://www.microimages.com/documentation/Tutorials/hyprssec.pdf>
- [2] P. Shippert (2003), Introduction to hyperspectral image analysis, <http://spacejournal.ohio.edu/pdf/shippert.pdf>
- [3] J. Kutrašnik, M. Burmen, F. Pernuš, B. Likar (2009), Near-infrared imaging of subcutaneous veins, V: Zborniku ROSUS 2009, Maribor, Slovenija, str. 101–109.
- [4] Polarization imaging camera PI-100, http://www.photonic-lattice.com/en/Pol_Camera0.html
- [5] A. G. Andreou, Z. K. Kalayjian(2002), Polarization imaging: Principles and integrated polarimeters, IEEE Sensors journal, vol. 2, št. 6, str. 566–576.
- [6] L. Bigué, P. Ambs, A. Jaulin, A. Foulonneau, L. Gendre (2008), Dynamic polarimetric imaging: Overview and implementation using liquid crystal cells, Journal of Physics: Conference Series, vol. 139, št. 1, str. 012007.
- [7] How interferometers work: a simple introduction to interferometry, <http://www.explainthatstuff.com/howinterferometerswork.html>
- [8] Interferometry basics, http://www.cyanogen.com/help/quickfringe/...QUICK_FRINGE.htm#QUICK_FRINGEInterferometry_Basics.htm

3. Napredna predobdelava slik

Odstranjevanje pegastega šuma

Barvna vztrajnost

Odstranjevanje pegastega šuma

- ▷ Odstranjevanje/zmanjševanje šuma (*noise removal/reduction*) iz slik še vedno aktualen raziskovalni izziv:
 - ★ postopki vnašajo artefakte v slike
 - ★ zameglijo oz. zapacajo (*blur*) slike
- ▷ **Šum razdelimo v dve skupini:**
 - ★ šum, neodvisen od slikovnih podatkov
 - ★ šum, odvisen od slikovnih podatkov

A. Šum, neodvisen od slikovnih podatkov

- ▷ Modeliramo ga v obliki modela aditivnega šuma

$$I(\mathbf{p}) = X(\mathbf{p}) + N(\mathbf{p})$$

I	–	posneta oz. šumna digitalna slika
X	–	brezšumna oz. želeno digitalna slika
N	–	šum
$\mathbf{p} = [i, j]$	–	piksel

- ▷ Pri obravnavanju te vrste šuma običajno sprejmemo določene predpostavke:
 - ★ šum ima ničelno povprečje
 - ★ varianca šuma je σ_N^2
 - ★ šum je enakomerno porazdeljen po celotni frekvenčni domeni (t.i. **bel oz. Gaussov šum**)

Odstranjevanje pegastega šuma

(2)

B. Šum, odvisen od slikovnih podatkov

- ▷ Modeliramo ga v obliki multiplikativnega oz. nelinearnega modela šuma

$$I(\mathbf{p}) = X(\mathbf{p})N(\mathbf{p})$$

Statistične mere za merjenje vpliva šuma

- ▷ Uporabljene oznake:
 - ★ I – originalna oz. šumna slika, $\hat{X}$ – slika z odstranjenim šumom
 - ★ σ – standardni odklon, μ – povprečje v določenem oknu oz. celotni sliki
- ▷ **MSE** (*mean squared error*):

$$MSE = \frac{1}{MN} \sum_i \sum_j \left(I(\mathbf{p}) - \hat{X}(\mathbf{p}) \right)^2$$

- ▷ **RMSE** (*root mean squared error*):

$$RMSE = \sqrt{MSE}$$

- ▷ **PSNR** (*peak signal-to-noise ratio*):

$$PSNR = 20 \log_{10} \left(\frac{\max_{\mathbf{p}} (I(\mathbf{p}))}{RMSE} \right) \quad [\text{dB}]$$

Odstranjevanje pegastega šuma

(3)

- ▷ SNR (*signal-to-noise ratio*):

$$SNR = 10 \log_{10} \frac{\sigma_I^2}{\sigma_{\hat{X}}^2} \quad [\text{dB}]$$

$$SNR = \frac{\mu_I}{\sigma_I}$$

- ▷ S/MSE (*signal-to-mean-square error ratio*):

$$S/MSE = 10 \log_{10} \frac{\sum_i \sum_j \hat{X}(\mathbf{p})^2}{\sum_i \sum_j \left(I(\mathbf{p}) - \hat{X}(\mathbf{p}) \right)^2} \quad [\text{dB}]$$

Primer:



MSE = 902,7 ; SNR = 2,0 dB

PSNR = 18,6 dB



MSE = 3074,3 ; SNR = 1,5 dB

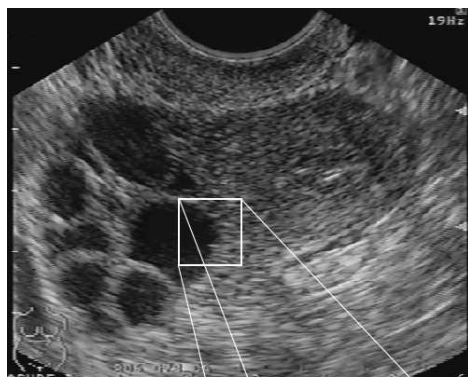
PSNR = 13,3 dB

Odstranjevanje pegastega šuma

(4)

Pegasti šum (*speckle noise*)

- ▷ Šum, odvisen od slikovnih podatkov: multiplikativen šum.
- ▷ Artefakt, ki ga povzroči interferenca energije iz naključno porazdeljenega raztresanja (*scattering*).
- ▷ Pojavlja se pri vsakem koherentnem slikanju (npr. ultrazvočnih slikovnih materialih, radarskem slikanju).
 - ★ sliko pri koherentnem slikanju dobimo kot seštevek odbitih signalov (valovanj) od razsipovalcev (*scatterers*).
- ▷ Sliki daje karakterističen zrnat izgled.



medicinska ultrazvočna slika



slika SAR

Odstranjevanje pegastega šuma

(5)

Kako nastane?

1. Znotraj ene ločljivostne celice množica elementarnih razsipovalcev odbije vpadno valovanje proti senzorju.
 2. Nazaj razsipano koherentno valovanje z različnimi fazami je izpostavljeno konstruktivni ali destruktivni interferenci na naključen način.
- ▷ Pegasti šum se pojavi, ko se valovanje razsipa od površine, katere hrapavost je reda valovne dolžine valovanja, ki zatem povzroči interferenco.
 - ▷ **Statistične lastnosti tega šuma so odvisne od gostote in prostorske razporeditve razsipovalcev:**
 - ★ pega ima naključno in deterministično naravo – razsipovalci so namreč porazdeljeni naključno oz. koherentno na površini (tkivu)
 - ★ **tekstura opazovane pege (*speckle*) ne ustreza spodaj ležečemu objektu**
 - ▷ lokalna svetlost vzorca pa kljub temu odseva lokalno ehogeničnost spodaj ležečih razsipovalcev

Odstranjevanje pegastega šuma

(6)

Metode za odstranjevanje/zmanjševanje pegastega šuma

- ▷ Primerna metoda za odstranjevanje te vrste šuma mora:
 - ★ izboljšati razmerje SNR
 - ★ ohraniti robove in linije v slikah

Uporabljene oznake (prvi del je prikazan na str. 62)

- ▷ Predpostavka: šum ima povprečje 1 ($\mu_N = 1$)

$$ENL = \left(\frac{\mu_I}{\sigma_I} \right)^2 = \frac{1}{\sigma_N} = L$$

- ▷ ENL (*equivalent number of looks*) – za ocenjevanje nivoja pegastega šuma v slikah SAR

A. Adaptivni mediani filter (*adaptive neighbourhood median filter*)

Postopek:

1. Definiramo velikost večje maske (okna, jedra) H_1 in manjše maske H_2 .
2. Vse piksle, ki so manjši od praga T , filtriramo z medianim filtrom z masko H_1 , preostale piksle pa z medianim filtrom z masko H_2 .
3. Korak 2 lahko ponovimo $2\times$.

Odstranjevanje pegastega šuma

(7)

B. Kuanov filter

- ▷ Temelji na kriteriju minimuma povprečne kvadrirane napake (*MMSE – minimum square error*).
- ▷ Predpostavlja naslednji multiplikativen model šuma:

$$I(\mathbf{p}) = X(\mathbf{p}) + (N(\mathbf{p}) - 1)X(\mathbf{p})$$

- ▷ Tipična uporabljena predpostavka:
 - ★ šum ima povprečje enako 1 (*unit-mean noise assumption*)
- ▷ Postopek:

$$\hat{X}(\mathbf{p}) = \mu_I + \frac{\sigma_X^2 (I(\mathbf{p}) - \mu_I)}{\sigma_X^2 + (\mu_I + \sigma_X^2) / L} \quad (1)$$

kjer

$$\sigma_X^2 = \frac{L\sigma_I^2 - \mu_I^2}{L + 1}$$

- ▷ V patološki situaciji, kadar je $\sigma_X^2 < 0$, določimo $\hat{X}(\mathbf{p}) = \mu_I$.

C. Leejev (MMSE) filter

- ▷ Posebna oblika Kuanovega filtra, kjer iz enačbe (1) odstranimo člen σ_X^2 / L .

D. Filter gama

- ▷ Je filter MAP (*maximum a posteriori*), ki temelji na Bayesovi statistični analizi slike.
- ▷ Predpostavka:
 - ★ odbojnost radarskih žarkov, kakor tudi pegasti šum sta porazdeljena po porazdelitvi gama. Superpozicija obeh porazdelitev vrne porazdelitev K.
- ▷ Postopek:

$$\hat{X}(\mathbf{p}) = \frac{(\alpha - L - 1) \mu_l + \sqrt{\mu_l (\alpha - L - 1)^2 + 4\alpha L \mu_l l(\mathbf{p})}}{2\alpha}$$

kjer

$$\alpha = \frac{L + 1}{L (\sigma_l / \mu_l)^2 - 1}$$

E. Frostov filter

- ▷ Je adaptivni Wienerjev filter, ki opravi konvolucijo vrednosti pikslov znotraj okna (maske) fiksne velikosti z eksponentnim enotnim odzivom h , definiranim kot

$$h = e^{-KC_I(\mathbf{t}_0)\|\mathbf{t}\|}, \quad C_I = \frac{\sigma_I}{\mu_I},$$

- ▷ kjer je K parameter filtra (npr. $K = 1$), $\mathbf{t}_0$ predstavlja lokacijo obdelovanega piksla in $\|\mathbf{t}\|$ je razdalja, merjena od piksla $\mathbf{t}_0$ (oz. od središčnega piksla maske).
- ▷ **Za vsak piksel v sliki dejansko določimo svojo masko!**

F. Oddyjev filter

- ▷ Neke vrste nizko sito, ki spreminja velikost okna glede na lokalne statistike.
- ▷ Postopek:

$$\begin{aligned} \star \quad \hat{X}(\mathbf{p}) &= \mu_I, \text{ če } m < \alpha\mu_I \\ \star \quad \hat{X}(\mathbf{p}) &= \frac{\sum_k \sum_l W_{kl} I(k,l)}{\sum_k \sum_l W_{kl}}, \text{ sicer} \end{aligned}$$

kjer

$$\begin{aligned} \star \quad W_{kl} &= 1, \text{ če } |I(k,l) - I(\mathbf{p})| \leq m \\ \star \quad W_{kl} &= 0, \text{ sicer} \end{aligned}$$

Odstranjevanje pegastega šuma

(10)

- ▷ $\hat{X}$ ocenjujemo lokalno, v oknu velikosti 3×3 piksele:

$$m = \frac{1}{8} \sum_k \sum_l |I(k, l) - I(\mathbf{p})|$$

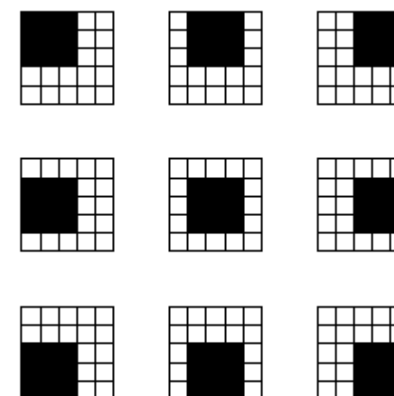
- ▷ W ima vlogo adaptivne binarne maske.
- ▷ α je parameter filtra (npr. $\alpha = 1$)

G. Filter AFS (adaptive filter on surfaces)

- ▷ Je filter z adaptivno masko, ki uporablja koncept vrednost l.e.s. (*local emerging surface*).
- ▷ **Vrednost l.e.s.** je ploščina ploskve slikovnega grafa, definiranega znotraj maske oz. okna.

- ▷ Postopek:

1. Vrednost l.e.s. izračunamo za 9 prikazanih binarnih mask velikosti 5×5 pikslov.
2. Izberemo masko z minimalno vrednostjo l.e.s.
3. Filtriramo z nizkim sitom z izbrano masko.
4. Dobljeno vrednost priredimo središčnemu pikslu okna, velikosti 5×5 pikslov.



Vir: [1]

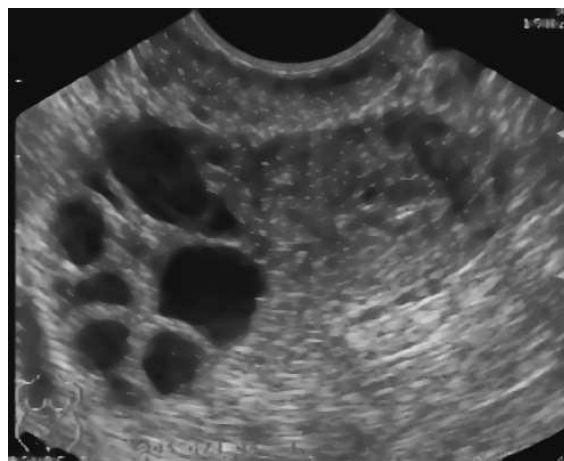
H. Drugi pristopi

- ▷ Odstranjevanje šuma z valčno transformacijo.
- ▷ Odstranjevanje šuma z nevronskimi mrežami (NN)
 - ★ Možen pristop:
 - ▷ Definiramo 3 slojni zvezni perceptron – $K \times L$ nevronov v 1. plasti, $K \times L$ nevronov v 2. plasti in 1 nevron v izhodni plasti
 - ▷ Vhod: seznam vrednosti pikslov iz maske, velikosti $K \times L$ pikslov
 - ▷ Izhod: filtrirana vrednost za središčni piksel maske
 - ▷ **Potrebno je učenje!**

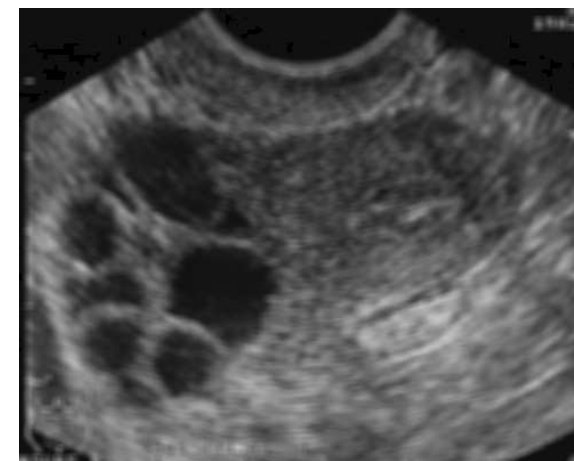
Primer:



original



Adaptivni median ($2 \times$)
 $H_1 = 9 \times 9$ in $H_2 = 3 \times 3$
PSNR = 25,45 dB



Kuanov filter
maska 7×7
PSNR = 23,19 dB

Barvna vztrajnost

- ▷ Barvna kalibracija ali pravilneje barvna vztrajnost (ang. colour constancy) omogoča konsistentno dojetje barve predmetov pod različnimi osvetlitvami.



Ali so na sliki pravilne barve?

- ▷ Barvna vztrajnost dobro razvita pri ljudeh, opazili pa jo tudi pri zlatih ribicah in čebelah.

Primer:

- ▷ Opazovanje predmeta pri sončni svetlobi:
 - ★ sončna svetloba je intenzivna, saj ima barvno temperaturo okrog 6000 K.
- ▷ Enak predmet opazujemo v stanovanju pod nitno žarnico (barvna temperatura med 2700 K do 3300 K):
 - ★ praktično ne opazimo spremembe v barvi
- ▷ Enak učinek pod svetlobo varčnih žarnic (barvna temperatura okrog 5500 K)
- ▷ V ekstremnih pogojih pa tudi barvna vztrajnost odpove (npr. v nekaterih nočnih klubih, kjer uporabljajo barvne filtre).

Vrste algoritmov za barvno vztrajnost

- ▷ Razvitih že mnogo algoritmov barvne vztrajnosti:
 - ★ najpogostejše uporabljene predpostavke: določen tip osvetlitve, karakteristika tipa površin, pomen povprečne barve v sliki ipd.
- ▷ Ločimo dve večji skupini algoritmov:
 - ★ algoritmi, ki predpostavljajo uniformno osvetlitev scene
 - ★ algoritmi, ki predpostavljajo več izvorov svetlobe in neuniformno osvetlitev

A. Algoritmi, ki predpostavljajo uniformno osvetlitev scene

- ▷ Predpostavka uniformne osvetlitve predvideva sceno z enim virom svetlobe, brez vidnih senc in brez sprememb v jakosti osvetlitve v prostoru.
- ▷ V primeru izpolnjevanja te predpostavke so zelo uspešni.
- ▷ Takšno osvetlitev je težavno pripraviti že v laboratoriju, v nekontroliranem okolju te omejitve skoraj nikoli ne izpolnimo:
 - ★ kljub temu se uporabljajo tudi za uravnavanje bele barve v naravnih scenah.

B. Algoritmi, ki predpostavljajo več izvorov svetlobe in neuniformno osvetlitev

- ▷ So splošnejši, a še vedno ne univerzalni.
- ▷ Pogosta predpostavka: počasno spreminjanje oz. nizek gradient osvetlitve.
- ▷ Ob tem se predpostavijo še hitre spremembe oz. visok gradient odbojnosti predmetov in geometrijskega faktorja scene (tj. vpliv prostorskih razmerij med virom svetlobe, odbojno površino in senzorjem).

Model za formiranje barvnih vrednosti piksla

- ▷ Barvna vrednost piksla je signal treh različnih vrst senzorjev, kjer se vsak senzor odziva na določen del vidnega spektra svetlobe.
 - ★ Odziv senzorja označimo s funkcijo spektralnega odziva $\mathcal{S}_v(\lambda)$, kjer indeks v označuje senzor oz. barvni kanal (tj. R, G ali B).
 - ★ Svetlobo, na senzorju na lokaciji piksla $\mathbf{p}$, označimo s funkcijo $E(\lambda, \mathbf{p})$.
 - ★ Vrednost signala oz. barvnega kanala piksla $C_v(\mathbf{p})$ modeliramo kot:

$$C_v(\mathbf{p}) = \int_{\lambda} \mathcal{S}_v(\lambda) E(\lambda, \mathbf{p}) d\lambda$$

- ★ Funkcija $E(\lambda, \mathbf{p})$ odvisna od
 - ▷ spektra svetlobe $L(\lambda, \mathbf{p})$, s katero je osvetljena scena,
 - ▷ odbojnosti predmetov $R(\lambda, \mathbf{p})$ v sceni in
 - ▷ vpadnega kota ter tipa odboja svetlobe od predmetov.
- ▷ Enačbo poenostavimo, če predpostavimo, da
 1. prisoten le razpršen odboj svetlobe – tj. za vse površine velja Lambertov kosinusni zakon.
 2. senzor se odziva samo na valovno dolžino λ_v – funkciji R in L sta odvisni od vrste senzorja:

$$C_v(\mathbf{p}) = G(\mathbf{p}) R_v(\mathbf{p}) L_v(\mathbf{p})$$

- ★ Funkcija $G(\mathbf{p})$ predstavlja geometrijski faktor scene.

A. Algoritem WPR (*white patch retinex*)

- ▷ Preprost algoritem, ki sliko popravlja po posameznih barvnih kanalih, in sicer neodvisno enega od drugega.
- ▷ Uporabljene predpostavke:
 1. osvetlitev scene je uniformna:
 - ★ osvetlitev tedaj neodvisna od položaja piksla.
 2. v sceni je prisoten predmet bele barve, ki odbija vso osvetlitev.
 - ★ takšen predmet ima maksimalno barvno vrednost, vsebuje pa tudi popolno informacijo o osvetlitvi, kar modeliramo kot:

$$L = \max (C(\mathbf{p}))$$

- ▷ Novo vrednost piksla, $O(\mathbf{p})$, neodvisno od osvetlitve scene, določimo kot:

$$O(\mathbf{p}) = G(\mathbf{p})R(\mathbf{p}) = \frac{C(\mathbf{p})}{L}$$

B. Algoritem GWA (*gray world assumption*)

- ▷ Predpostavka:
 - ★ v sceni predmeti različnih barv, pri čemer so vse barve enako zastopane.
 - ★ osvetlitev določimo iz "povprečne barve" v sliki, z izračunanim povprečjem nato uravnavamo belo barvo.
- ▷ Postopek:

$$O(\mathbf{p}) = \frac{C(\mathbf{p})}{\bar{C}\varphi}$$

kjer

- ★ $\bar{C}$ – povprečna vrednost barvnega kanala
- ★ φ – poljubno nastavljen faktor povečave (npr. $\varphi = 2$)

C. Algoritem CCN (*comprehensive colour normalization*)

- ▷ Iterativen algoritem, kjer zaporedoma izvajamo dva koraka:
 1. Odstranjevanje geometrijskega faktorja s pomočjo barvnega normiranja:

$$O'(\mathbf{p}) = \frac{C(\mathbf{p})}{C_R(\mathbf{p}) + C_G(\mathbf{p}) + C_B(\mathbf{p})}$$

2. Skaliranje vrednosti z algoritmom GWA:

$$O(\mathbf{p}) = \frac{O'(\mathbf{p})}{\bar{C}_\varphi}$$

3. V naslednji iteraciji: $C(\mathbf{p}) \leftarrow O(\mathbf{p})$
4. Zaključni pogoj (konvergenca):
 - ★ vse vhodne vrednosti pikselov v trenutni iteraciji, $C(\mathbf{p})$, enake izhodnim vrednostim $O(\mathbf{p})$

- ▷ **Z barvno normalizacijo izgubimo informacijo o intenzivnosti za posamezen piksel:**
 - ★ Če to informacijo potrebujemo, potem uporabimo intenzivnost izvirne slike in jo vstavimo v obdelano sliko (npr. komponento Y originalne slike v prostoru YCbCr priredimo komponenti Y obdelani sliki).

D. Algoritem LSAC (*local space average colour*)

- ▷ Temelji na algoritmu GWA.
- ▷ Predpostavka: Imamo počasi spreminjajočo se osvetlitev.
- ▷ Osvetlitev izločimo iz slike z 2-D nizkim filtrom.
- ▷ Postopek:

$$O(\mathbf{p}) = \frac{C(\mathbf{p})}{L(\mathbf{p})} = \frac{C(\mathbf{p})}{\bar{C}(\mathbf{p})\varphi}$$

- ★ $\bar{C}(\mathbf{p})$ – funkcija, ki iz slike izloči visoke frekvence (tj. počasi spreminjajočo se osvetlitev):

$$\bar{C}(\mathbf{p}) = C(\mathbf{p}) * K(\mathbf{p})$$

- ★ $K(\mathbf{p})$ – jedro 2D nizkega sita (npr. Gaussovo ali eksponentno jedro)

Primer: Gaussovo jedro

$$K(\mathbf{p}) = K(x, y) = a^{-1} e^{-\frac{x^2 + y^2}{2\rho^2}}$$

- ▷ a – ojačanje filtra
- ▷ Optimalna vrednost parametra ρ se določi iz dimenzije slike kot

$$\rho = 0,093 \max(M, N)$$

E. Algoritem RSR (*random spray retinex*)

- ▷ Predpostavka:
 - ★ v sliki so prisotne bele površine, ki odbijajo celoten spekter osvetlitve (tj. vsebujejo popoln podatek o osvetlitvi).
 - ★ osvetlitev je počasi spreminjajoča se.
- ▷ Barvno vrednost piksla popravljamo s pomočjo P razpršenih množic ($p = 1, \dots, P$), vsaka izmed njih pa vsebuje K pikslov.
- ▷ k -ti piksel iz p -te razpršene množice, $\mathbf{p}_{p,k} = [i_{p,k}, j_{p,k}]$, določimo kot

$$\begin{aligned}i_{p,k} &= i + r\eta \cos(\mu) \\j_{p,k} &= j + r\eta \sin(\mu)\end{aligned}$$

kjer $\eta \in U(0, 1)$, $\mu \in U(0, 2\pi)$, $\mathbf{p} = [i, j]$ in r je radij razpršene množice.

- ▷ **Postopek barvnega izravnavanja:**

$$O(\mathbf{p}) = \frac{1}{P} \sum_{p=1}^P \frac{C(\mathbf{p})}{\max_k (C(\mathbf{p}_{p,k}))}$$

- ▷ Prosti parametri:
 - ★ radij razpršene množice pikslov, r (optimalno: velikost diagonale slike)
 - ★ število razpršenih množic na piksel, P (npr. $P = 20$)
 - ★ število pikslov v razpršeni množici, K (npr. $K = 500$)
 - ★ **Optimalna nastavitve parametrov odvisna od vsebine slike!**

Barvni kalibrator

- ▷ Barvni kalibrator oz. barvna tablica (*colour checker*) – referenčni predmet na digitalni sliki z znano barvo.
- ▷ Uporaba:
 - ★ Za barvno kalibracijo (npr. fotografiranje), tudi za ocenjevanje uspešnosti barvnega popravljanja.
- ▷ Izdelava: tablica z barvnimi polji, kjer imajo barve specifične lastnosti za širok nabor osvetlitev (skoraj neodvisne od osvetlitve)

- ▷ **Kalibrator GretagMacbeth oz. X-Rite**

- ★ Velikosti: 216×279 oz. 57×82 mm.
- ★ 1. in 2. vrstica: naravne barve in barve, ki jih je težko reproducirati.
- ★ 3. vrstica: osnovne barve seštevalnega prostora RGB in odštevalnega modela CMY.
- ★ 4. vrstica: sivinska lestvica.



- ▷ **Kalibrator QP201**

- ★ Velikosti: 142×40 mm.
- ★ Barve: Nima jasno razdeljenih skupin barv (razen sivinske lestvice)



Barvna vztrajnost

(10)

Primer: Povzeto po [2]



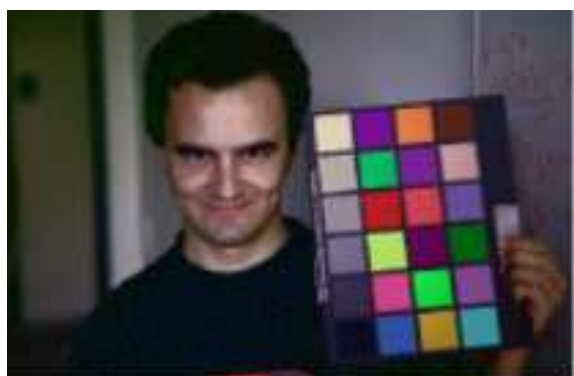
Originalna slika



Algoritem WPR



Algoritem GWA



Algoritem CCN



Algoritem LSAC



Algoritem RSR

- ▷ **Merjenje uspešnosti:**
 - ★ s pomočjo mer kot MSE, RMSE...

Dodatna literatura

- [1] L. Gagnon, A. Jouan (1997), Speckle Filtering of SAR Images - A Comparative Study Between Complex-Wavelet-Based and Standard Filters, V: Zborniku SPIE, vol. 3169, San Diego, ZDA, str. 80-91.
- [2] M. Šavc (2011), Primerjava algoritmov za barvno kalibriranje digitalnih slik, Diplomsko delo, UM-FERI, Maribor, Slovenija.
- [3] M. Sonka, V. Hlavac, R. Boyle (1994), Image processing, analysis and machine vision, Chapman & Hall Computing, London, VB.

4. Napredna segmentacija

Posplošena Houghova transformacija

Modeli aktivnih kontur – kače

Analiza neodvisnih komponent

Posplošena Houghova transformacija

- ▷ Pomembne informacije o objektu v sliki so pogostokrat skrite v obliki njegove meje (*boundary*):
 - ★ notranja in zunanja meja objekta (glej zapiske predavanj URVRV, poglavje 4)
- ▷ Houghova transformacija (HT) je metoda za detektiranje krivulj, pri čemer izkorišča dualnost med točkami krivulje in parametri te iste krivulje.
- ▷ Obstaja HT za detektiranje:
 - ★ analitičnih krivulj in
 - ★ neanalitičnih krivulj
- ▷ HT se opira na informacijo o:
 - ★ **robni piksli**
 - ▷ uporaba zgolj informacije o jakosti robnega piksla oz. gradienta
 - ★ **celotni sliki robov**
 - ▷ rob je vektorska spremenljivka; rob = [jakost roba, smer gradienta]
 - ▷ uporabljata se obe informaciji
- ▷ Pojmi: slika robov (*edge image*), robni piksel (*edge pixel*), jakost gradienta, smer gradienta (glej zapiske predavanj URVRV, poglavje 3).

A. HT za analitične krivulje

- ▷ Mejo iskanega objekta lahko predstavimo z analitično krivuljo.
- ▷ Splošno analitično krivuljo zapišemo v obliki:

kjer $\mathcal{F}(\mathbf{x}, \mathbf{a}) = 0$

- ★ $\mathbf{x}$ – slikovna točka oz. piksel
- ★ $\mathbf{a}$ – vektor parametrov krivulje

1. HT za iskanje linij oz. premic

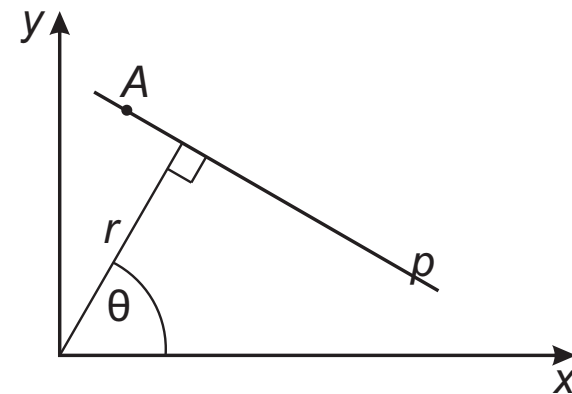
- ▷ Povzetek, podrobnosti v zapiskih predavanj URVRV, poglavje 4

- ▷ Klasična enačba premice:

$$y = kx + n$$

- ▷ Normalna oz. parametrična oblika enačbe:

$$x \cos \theta + y \sin \theta = r$$



- ▷ HT preslika točke iz kartezijske 2D-ravnine (tj. x, y) v parametrični prostor modela (tj. r, θ)

- ★ primer: točke premice p se preslikajo v isto točko v Houghovem prostoru

Posplošena Houghova transformacija

(3)

- ▷ HT apliciramo na binarni sliki (npr. binarizirani sliki robov)
- ▷ Za piksele v sliki pa seveda ne vemo, kakšna(e) premica(e) poteka(jo) skozi njih (če sploh!):
 - ★ skozi posamezen piksel lahko torej teoretično potekajo premice z vsemi možnimi smermi θ
- ▷ Piksel $\mathbf{p} = [i, j]$ se s HT preslika v sinusno krivuljo:
 - ★ Pri znani orientaciji, tj. θ , izračunamo oddaljenost r kot:

$$r = i \cos \theta + j \sin \theta \quad (2)$$

- ★ izračun naredimo pri vseh možnih orientacijah, rezultat pa shranimo v akumulatorsko polje $A(\theta, r)$
 - ▷ Houghov parametrični prostor in izračunane vrednosti moramo diskretizirati!

Postopek:

1. Inicializiraj akumulator A , tj. $A(\theta, r) = 0$
2. FOR vsak robni piksel
 - FOR $\theta = \min_{\theta} : \Delta\theta : \max_{\theta}$
 - Izračunaj enačbo (2)
 - $A(\theta, r) = A(\theta, r) + \Delta A$
3. Poišči lokalne maksimume v akumulatorju A , kar ustreza položajem premic v sliki.

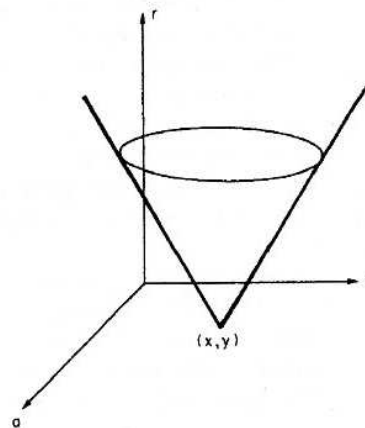
2. HT za iskanje krožnic, izključno na osnovi robnih pikslov

- ▷ Klasična enačba krožnice:

$$(x - a)^2 + (y - b)^2 = r^2$$

- ▷ Postavimo si naslednje vprašanje:

- ★ Če naj bi piksel $p = [x, y]$ ležal na krožnici, kaj je možen prostor parametrov (a, b, r) za to krožnico (v parametričnem prostoru)?



Odgovor: Pravilni krožni stožec (vir: [1]).

- ▷ Če v sliki dejansko obstaja krožnica, potem se bo množica takšnih pravilnih krožnih stožcev sekala v skupni točki.
- ▷ Potrebujemo 3D akumulator A, tj. $A(a, b, r)$.

Posplošena Houghova transformacija

(5)

Postopek:

1. Inicializiraj akumulator A, tj. $A(a, b, r) = 0$
2. FOR vsak robni piksel
 FOR $a = \min_a : \Delta a : \max_a$
 FOR $b = \min_b : \Delta b : \max_b$
 $r = \sqrt{(x - a)^2 + (y - b)^2}$
 $A(a, b, r) = A(a, b, r) + \Delta A$
3. Poišči lokalne maksimume v akumulatorju A, kar ustreza položajem krožnic v sliki.

DN:

- ▷ HT za iskanje elips, izključno na osnovi robnih pikslov

$$\frac{(x - x_0)^2}{a^2} + \frac{(y - y_0)^2}{b^2} = 1$$

- ▷ HT za iskanje elips, izključno na osnovi robnih pikslov, pri čemer je glavna os elipse rotirana za kot φ glede na os x

$$\begin{aligned}x(t) &= x_0 + a \cos t \cos \varphi - b \sin t \sin \varphi \\y(t) &= y_0 + a \cos t \sin \varphi + b \sin t \cos \varphi\end{aligned}$$

3. HT za analitične krivulje v sivinskih slikah, na osnovi vseh informacij iz slike robov

- ▷ Iz originalne slike moramo izračunati kompletno sliko robov:
 - ★ jakost oz. velikost gradienta (na tej osnovi določamo robne piksele!)
 - ★ smer gradienta ψ
- ▷ V HT uporabljamo tako originalno enačbo krivulje

$$\mathcal{F}(\mathbf{x}, \mathbf{a}) = 0 \quad (3)$$

kakor tudi njen odvod

$$\frac{d\mathcal{F}}{dx}(\mathbf{x}, \mathbf{a}) = 0 \quad (4)$$

- ▷ V odvodu dobimo člen $\frac{dy}{dx}$, ki je poznan, saj

$$\frac{dy}{dx} = \operatorname{tg}\left[\psi(\mathbf{x}) - \frac{\pi}{2}\right]$$

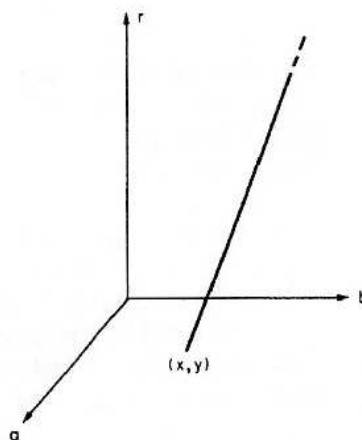
Postopek:

1. Inicializiraj akumulator A , tj. $A(\mathbf{a}) = 0$
2. FOR vsak robni piksel
 - Izračunaj vse parametre $\mathbf{a}$, za katere veljata enačbi (3) in (4)
 - $A(\mathbf{a}) = A(\mathbf{a}) + \Delta A$
3. Poišči lokalne maksimume v akumulatorju A .

Posplošena Houghova transformacija

(7)

- ▷ Če v HT uporabljamo še informacijo o smeri gradienta, se računska zahtevnost bistveno zmanjša.
- ▷ **Primer:**
 - ★ Pri iskanju krožnic s tem postopkom, se število prostih parametrov zmanjša na 1.
 - ★ Prostor možnih parametrov se iz stožca zreducira na premico.



Prostor možnih parametrov pri iskanju krožnic s pomočjo informacije o smeri gradienta.

4. HT za iskanje elips v sivinskih slikah, s pomočjo informacije o smeri gradienta

▷ Začetna predpostavka:

- ★ elipsa orientirana tako, da je njena glavna os vzporedna z x -osjo (v nadaljevanju bomo dodali še rotacijo!)

▷ Enačba elipse:

$$\frac{(x - x_0)^2}{a^2} + \frac{(y - y_0)^2}{b^2} = 1 \quad (5)$$

▷ Vpeljimo $X = x - x_0$ in $Y = y - y_0$ ter odvajamo po X . Dobimo

$$\frac{2X}{a^2} + \frac{2Y}{b^2} \frac{dY}{dX} = 0 \quad (6)$$

▷ Vrednost $\frac{dY}{dX}$ je poznana iz informacije o robnem pikslu:

- ★ označimo:

$$\frac{dY}{dX} = \xi$$

▷ Iz enačb (5) in (6) izračunamo

$$Y = \pm \frac{b^2}{\sqrt{1 + \frac{a^2}{b^2} \xi^2}} \quad X = \pm \frac{a^2}{\sqrt{1 + \frac{b^2}{a^2} \xi^2}} \quad (7)$$

Posplošena Houghova transformacija

(9)

- ▷ Torej: Če poznamo a, b, x, y in $\frac{dY}{dX}$ potem določimo x_0 in y_0 kot

$$y_0 = y \pm \frac{b^2}{\sqrt{1 + \frac{a^2}{b^2} \xi^2}} \quad x_0 = x \pm \frac{a^2}{\sqrt{1 + \frac{b^2}{a^2} \xi^2}} \quad (8)$$

- ★ Dobimo štiri rešitve v štirih kvadrantih.
- ★ Pravilen kvadrant izberemo na osnovi testiranja predznaka izrazov dY in dX

- ▷ Upoštevajmo še rotacijo elipse, tj. parameter φ

- ★ Parameter ξ se sedaj določi kot

$$\xi = \operatorname{tg}(\psi - \varphi - \frac{\pi}{2}) \quad (9)$$

- ★ Z izračunanim ξ določimo X in Y
- ★ Na koncu dobljeno točko (X, Y) rotiramo za kot φ , s čimer dobimo središče elipse (x_0, y_0)

Posplošena Houghova transformacija

(10)

Postopek:

1. Inicializiraj akumulator A , tj. $A(x_0, y_0, \varphi, a, b) = 0$
2. FOR vsak robni piksel
 - FOR $a = \min_a : \Delta a : \max_a$
 - FOR $b = \min_b : \Delta b : \max_b$
 - FOR $\varphi = \min_\varphi : \Delta\varphi : \max_\varphi$
 - Izračunaj enačbo (9).
 - Določi X in Y po enačbi (7), pri čemer določi predznak na osnovi opazovanja dY in dX .
 - Rotiraj X in Y za kot φ .
 - $x_0 = x + X$
 - $y_0 = y + Y$
 - $A(x_0, y_0, \varphi, a, b) = A(x_0, y_0, \varphi, a, b) + \Delta A$
3. Poišči lokalne maksimume v akumulatorju A , kar ustreza položajem elips v sliki.

B. Posplošitev HT za neanalitične krivulje

- ▷ Za opis oblike neanalitične krivulje uporabljamo naslednji vektor parametrov

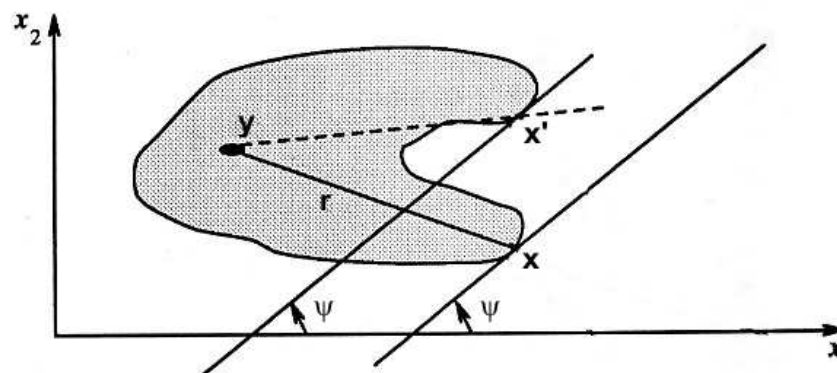
$$\mathbf{a} = [\mathbf{y}, \mathbf{s}, \varphi]$$

kjer

- ★ $\mathbf{y} = [x_0, y_0]$ – referenčno izhodišče za obliko (krivuljo)
 - ★ φ – orientacija krivulje
 - ★ $\mathbf{s} = [s_x, s_y]$ – ortogonalna skalirna faktorja (v praksi $s_x = s_y$)
 - ★ Položaj referenčnega izhodišča $\mathbf{y}$ opišemo v obliki tabele možnih orientacij robnih pikslov (tj. R-tabela)
- ▷ Ključna stvar pri posplošitvi HT je uporaba informacije o smeri gradienta:
 - ★ ne le, da pohitri algoritem, ampak tudi bistveno izboljša natančnost!

1. R-tabela

- ▷ R-tabelo skonstruiramo v učni fazi za vsako iskano obliko.



Postopek:

1. Izberi referenčno točko y znotraj iskane oblike.
2. FOR vsako točko x iz meje
Izračunaj smer gradienta $\psi(x)$.
 $r = y - x$
Shrani r v R-tabelo v odvisnosti od $\psi(x)$.

Primer: Oblika R-tabele [1]

i	ψ_i	R_{ψ_i}
0	0	$\{r y - r = x \wedge \psi(x) = 0 \wedge x \text{ je na meji}\}$
1	$\Delta\psi$	$\{r y - r = x \wedge \psi(x) = \Delta\psi \wedge x \text{ je na meji}\}$
2	$2\Delta\psi$	$\{r y - r = x \wedge \psi(x) = 2\Delta\psi \wedge x \text{ je na meji}\}$
...	...	...

Lastnosti R-tabele

Za iskano obliko poznamo njeno R-tabelo, tj. $R(\psi)$:

▷ Kakšna je nova R-tabela, $R'(\psi)$, pri znani transformaciji?

1. Skaliranje oblike (objekta) za faktor s

$$R'(\psi) = sR(\psi) \quad (10)$$

2. Rotiranje oblike (objekta) za kot φ

$$R'(\psi) = \text{Rotacija}\{R[(\psi - \varphi) \bmod 2\pi], \varphi\} \quad (11)$$

- ▷ vse indekse najprej inkrementiramo za $(-\varphi \bmod 2\pi)$
- ▷ dobljene vektorje $\mathbf{r}$ pa rotiramo za kot φ

3. Sprememba referenčne točke iz $\mathbf{y}$ v $\mathbf{y}'$

$$R'(\psi) = R(\psi) + (\mathbf{y} - \mathbf{y}')$$

2. Iskanje poljubnih oblik

1. Inicializiraj akumulator A , tj. $A(\mathbf{y}, s, \varphi) = 0$
2. FOR vsak robni piksel $\mathbf{x}$
 FOR $s = \min_s : \Delta s : \max_s$
 FOR $\varphi = \min_\varphi : \Delta \varphi : \max_\varphi$
 Določi R' po enačbi (10).
 Vstavi R' v enačbo (11) in določi R'' .

 FOR $\mathbf{r} \in R''(\psi(\mathbf{x}))$
 $A(\mathbf{x} + \mathbf{r}, s, \varphi) = A(\mathbf{x} + \mathbf{r}, s, \varphi) + \Delta A$
3. Poišči lokalne maksimume v akumulatorju A .

▷ Učinkovite implementacije posplošene HT in njene variante najdemo v [2].

C. Inkrementalne strategije in kompenziranje napak

1. Inkrementalne strategije

- ▷ Če v vsaki iteraciji inkrementiramo akumulator za 1, potem je vsebina akumulatorja proporcionalna obsegu iskane oblike:
 - ★ takšna strategija preferira iskanje oblik, kjer je velik del objekta (krivulje) možno detektirati
- ▷ Če želimo detektirati krajše, zelo pomembne dele objekta (npr. v primeru delnega zakrivanja objektov), potem so primernejše alternativne inkrementalne strategije:

$$A(\mathbf{a}) = A(\mathbf{a}) + |\text{grad}(\mathbf{x})|$$

ali pa kombinacija obojega

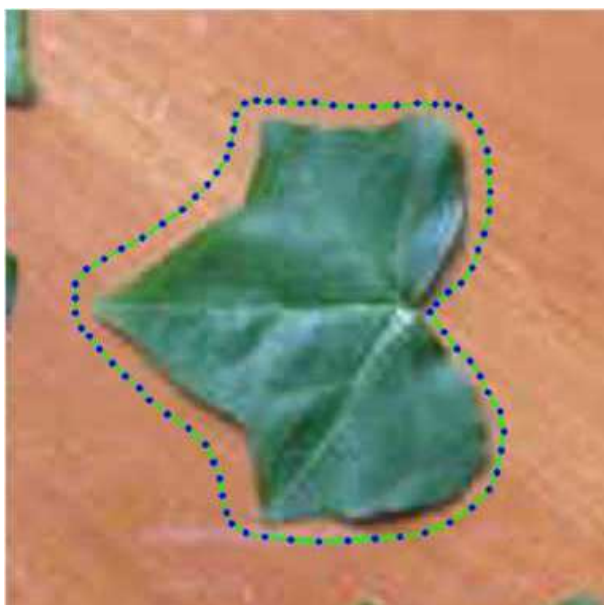
$$A(\mathbf{a}) = A(\mathbf{a}) + |\text{grad}(\mathbf{x})| + \text{konstanta}$$

2. Detektiranje lokalnih maksimumov

- ▷ Pri detektiranju maksimumov v akumulatorju A se lahko pojavijo problemi:
 - ★ mnogi faktorji vplivajo na natančnost izračunanega vektorja $\mathbf{a}$
 - ▷ **Posledica: Ob idealni točki $\mathbf{a}$ inkrementiramo v akumulatorju še množico lokacij v neposredni okolici te točke!**
- ▷ Pokazalo se je [1], da z glajenjem akumulatorja odpravimo nenatančnosti pri izračunu smeri gradienta in vektorja $\mathbf{r}$.

Modeli aktivnih kontur – kače

- ▷ Kača je spremenljiv zlepek (*deformable spline*) na osnovi minimizacije energije, na katero vplivajo omejitve in slikovne sile, ki jo povlečejo proti meji objekta.
- ▷ Kače so poseben primer splošne tehnike **ujemanja spremenljivega modela** (*matching of deformable model*) na sliko na osnovi minimizacije energije.



Začetna kača iz 78 kontrolnih točk.



Končno stanje kače po 128 iteracijah (vir: [4]).

A. Ideja osnovnega postopka kač

- ▷ Osnovni model kač je **kontroliran nepretrgan zlepek** (*controlled continuity spline*) pod vplivom slikovnih sil in sil zunanjih omejitev (*external constraint forces*):
 - ★ **interne sile zlepka** (*internal spline forces*) vnašajo omejitev gladkosti za posamezne odseke (*piecewise smoothness constraint*)
 - ★ **slikovne sile** potisnejo kačo proti pomembnim (*salient*) značilnicam:
 - ▷ npr. linijam, robovom, vnaprej pripravljenim konturam
 - ★ **sile zunanjih omejitev** potegnejo kačo proti želenim lokalnim minimumom:
 - ▷ npr. lahko jih posreduje uporabnik ali pa so rezultat višje nivojskih interpretacij
- ▷ Položaj kače predstavimo parametrično kot

$$\mathbf{v}(s) = [x(s), y(s)]$$

- ★ kjer je parameter s proporcionalen dolžini krožnega loka, tj. $s \in [0, 1]$

- ▷ **Energijski funkcional kače, E_{snake} , definiramo kot**

$$E_{snake} = \int_0^1 \left(E_{int}(\mathbf{v}(s)) + E_{image}(\mathbf{v}(s)) + E_{con}(\mathbf{v}(s)) \right) ds \quad (12)$$

- ★ E_{int} – interna energija zlepka zaradi upogibanja
- ★ E_{image} – slikovne sile
- ★ E_{con} – sile zunanjih omejitev

B. Interna energija

- ▷ Definirana je kot

$$E_{int} = \frac{1}{2} \left(\alpha(s) |\mathbf{v}_s(s)|^2 + \beta(s) |\mathbf{v}_{ss}(s)|^2 \right)$$

kjer

- ★ $\mathbf{v}_s$ – prvi odvod $\mathbf{v}$ po s (tj. člen 1. reda)
 - ★ $\mathbf{v}_{ss}$ – drugi odvod $\mathbf{v}$ po s (tj. člen 2. reda)
 - ★ α in β – uteži
- ▷ Člen 1. reda povzroči, da se kača obnaša kot membrana.
 - ▷ Člen 2. reda povzroči, da se kača obnaša kot tanek metalni papir (*thin-plate*).
 - ▷ Uteži $\alpha(s)$ in $\beta(s)$ kontrolirata pomembnost obeh členov:
 - ★ velikokrat: $\alpha(s) = \alpha$ in $\beta(s) = \beta$
 - ★ če $\beta(s) = 0$ pri določenem vozlišču (točki), potem dovolimo, da tam kača ustvari vogal (*corner*).

C. Slikovne sile

- ▷ To je energijski funkcional, ki priteguje kačo k pomembnim značilnostim v sliki.
- ▷ Skupna slikovna energija je definirana kot utežena kombinacija treh energijskih funkcionalov:

$$E_{image} = w_{line}E_{line} + w_{edge}E_{edge} + w_{term}E_{term} \quad (13)$$

kjer

- ★ E_{line} – energijski funkcional, ki pritegne kačo k linijam
- ★ E_{edge} – energijski funkcional, ki pritegne kačo k robovom
- ★ E_{term} – energijski funkcional, ki pritegne kačo h zaključkom
- ★ w_{line} , w_{edge} in w_{term} – uteži

1. Linijski funkcional

$$E_{line} = I(x, y)$$

- ▷ V odvisnosti od predznaka w_{line} bomo kačo pritegnili bodisi k svetlim oz. temnim linijam v sliki.

2. Robni funkcional

$$E_{edge} = -|\nabla I(x, y)|^2$$

- ▷ V tem primeru kačo pritegujejo konture z velikimi slikovnimi gradienti.

3. Funkcional zaključkov

- ▷ Za iskanje zaključkov linijskih segmentov in oglišč se uporablja ukrivljenost (*curvature*) v nivojskih linijah v rahlo zglajeni sliki C.
- ▷ Zglajena slika je definirana kot

$$C(x, y) = G_{\sigma}(x, y) * I(x, y)$$

kjer je G_{σ} Gaussov filter.

- ▷ **Energijski funkcional je definiran kot**

$$E_{term} = \frac{C_{yy}C_x^2 - 2C_{xy}C_xC_y + C_{xx}C_y^2}{(C_x^2 + C_y^2)^{\frac{3}{2}}} \quad (14)$$

kjer so C_x , C_y , C_{xx} , C_{yy} in C_{xy} ustrezni parcialni odvodi

- ▷ S kombiniranjem funkcionalov E_{edge} in E_{term} lahko kreiramo kače, ki jih pritegnejo robovi oz. zaključki.

D. Postopek minimizacije energijskega funkcionala kače

- ▷ Postopek je iterativna tehnika, ki uporablja redke matrike, pri čemer je časovna zahtevnost reda $\mathcal{O}(n)$.
- ▷ V vsaki iteraciji učinkovito izvedemo implicitni Eulerjev korak v odvisnosti od interne energije ter eksplicitni Eulerjev korak v odvisnosti od slikovne energije in energije zunanjih omejitev.
- ▷ Najprej energijo kače (enačba (12)) diskretiziramo

$$E_{snake} = \sum_{i=1}^n E_{int}(i) + E_{ext}(i)$$

- ★ parcialne odvode določimo s pomočjo razlik
- ★ i -to vozlišče kače $\mathbf{v}$ pa zapišemo v vektorski notaciji kot

$$\mathbf{v}_i = [x_i, y_i] = [x(ih), y(ih)]$$

- ▷ **Energijski funkcional E_{int} sedaj zapišemo kot**

$$E_{int}(i) = \alpha_i \frac{|\mathbf{v}_i - \mathbf{v}_{i-1}|^2}{2h^2} + \beta_i \frac{|\mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}|^2}{2h^4}$$

kjer $\mathbf{v}_0 = \mathbf{v}_n$

Modeli aktivnih kontur – kače

(7)

- ▷ Naj bo $\mathcal{F}_x(i) = \frac{\partial E_{ext}}{\partial x_i}$ in $\mathcal{F}_y(i) = \frac{\partial E_{ext}}{\partial y_i}$.
- ▷ Pripadajoča Eulerjeva enačba je

$$\begin{aligned} \alpha_i(\mathbf{v}_i - \mathbf{v}_{i-1}) - \alpha_{i+1}(\mathbf{v}_{i+1} - \mathbf{v}_i) + \beta_{i-1}(\mathbf{v}_{i-2} - 2\mathbf{v}_{i-1} + \mathbf{v}_i) \\ - 2\beta_i(\mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}) + \beta_{i+1}(\mathbf{v}_i - 2\mathbf{v}_{i+1} + \mathbf{v}_{i+2}) \\ + (\mathcal{F}_x(i), \mathcal{F}_y(i)) = 0 \end{aligned}$$

- ▷ Kar zapišemo v matrični notaciji kot

$$\begin{aligned} \mathbf{A}\mathbf{x} + \mathcal{F}_x(\mathbf{x}, \mathbf{y}) &= 0 \\ \mathbf{A}\mathbf{y} + \mathcal{F}_y(\mathbf{x}, \mathbf{y}) &= 0 \end{aligned} \tag{15}$$

kjer sta $\mathbf{x}$ in $\mathbf{y}$ n -dimenzionalna vektorja x oz. y koordinat vozlišč kače.

Modeli aktivnih kontur – kače

(8)

▷ Matrika A je pentadiagonalna matrika, velikosti $n \times n$, oblike

$$A = \begin{bmatrix} c & b & a & 0 & 0 & 0 & 0 & \dots & 0 & a & b \\ b & c & b & a & 0 & 0 & 0 & \dots & 0 & a \\ a & b & c & b & a & 0 & 0 & \dots & 0 & 0 \\ 0 & a & b & c & b & a & 0 & \dots & 0 \\ \vdots & & & & & & & & & \\ \vdots & & & & & & & & & \\ 0 & & & & & & \dots & a & b & c & b & a \\ a & 0 & & & & & \dots & 0 & a & b & c & b \\ b & a & 0 & & & & \dots & 0 & 0 & a & b & c \end{bmatrix}$$

kjer

- ★ $a = \beta$
- ★ $b = -(\alpha + 4\beta)$
- ★ $c = 2\alpha + 6\beta$
- ★ Zgoraj navedeno velja ob predpostavki, da $\alpha_i = \alpha$ in $\beta_i = \beta$

- ▷ Enačbo (15) rešimo kot

$$\begin{aligned}\mathbf{x}_t &= (\mathbf{A} + \gamma \mathbf{I})^{-1} (\gamma \mathbf{x}_{t-1} - \mathcal{F}_x(\mathbf{x}_{t-1}, \mathbf{y}_{t-1})) \\ \mathbf{y}_t &= (\mathbf{A} + \gamma \mathbf{I})^{-1} (\gamma \mathbf{y}_{t-1} - \mathcal{F}_y(\mathbf{x}_{t-1}, \mathbf{y}_{t-1}))\end{aligned}\tag{16}$$

kjer

★ γ – velikost koraka (pogosto $\gamma = 1$)

- ▷ Enačba (16) definira spremembo položaja vozlišč kače iz iteracije $t - 1 \rightarrow t$

E. Algoritem kač

1. Vnesi začetni približek (krivuljo) iskanega objekta.
2. Enakomerno razporedi n vozlišč po krivulji.
3. Določi E_{ext} po enačbi (13).
4. Izračunaj $\mathcal{F}_x = \frac{\partial E_{ext}}{\partial x}$ in $\mathcal{F}_y = \frac{\partial E_{ext}}{\partial y}$.
5. Določi matriko $\mathbf{A}$.
6. WHILE (ni izpolnjen konvergenčni pogoj)
 Določi nov položaj vozlišč kače v iteraciji t po enačbi (16).
 $t = t + 1$;
 Enakomerno razporedi n vozlišč po krivulji.

Opombe:

- ▷ V algoritmu imamo vse skozi opravka z n vozlišči:
 - ★ Če želimo število vozlišč spremeniti, moramo matriko A ponovno določiti!
- ▷ Konvergenčni pogoj:
 - ★ število iteracij, absolutni premik vozlišč $< T$ itd.
- ▷ Če želimo v posamezni iteraciji prirediti zgolj izbranemu vozlišču določene omejitve, potem moramo prav tako ustrezno ažurirati matriko A .

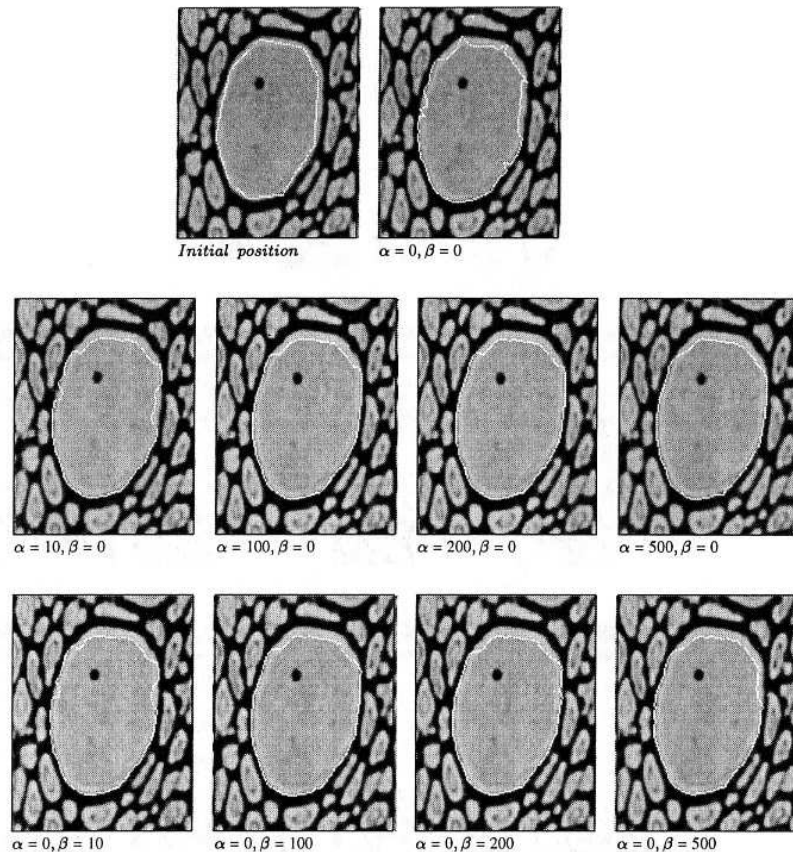
Problemi:

- ▷ Inicializacija začetnega položaja:
 - ★ ročni posegi, rezultat drugih metod...
- ▷ Določitev ustreznih vrednosti za parametre (α , β , w_{line} in w_{edge} ...)
 - ★ Parametri imajo ogromen vpliv na končni rezultat!

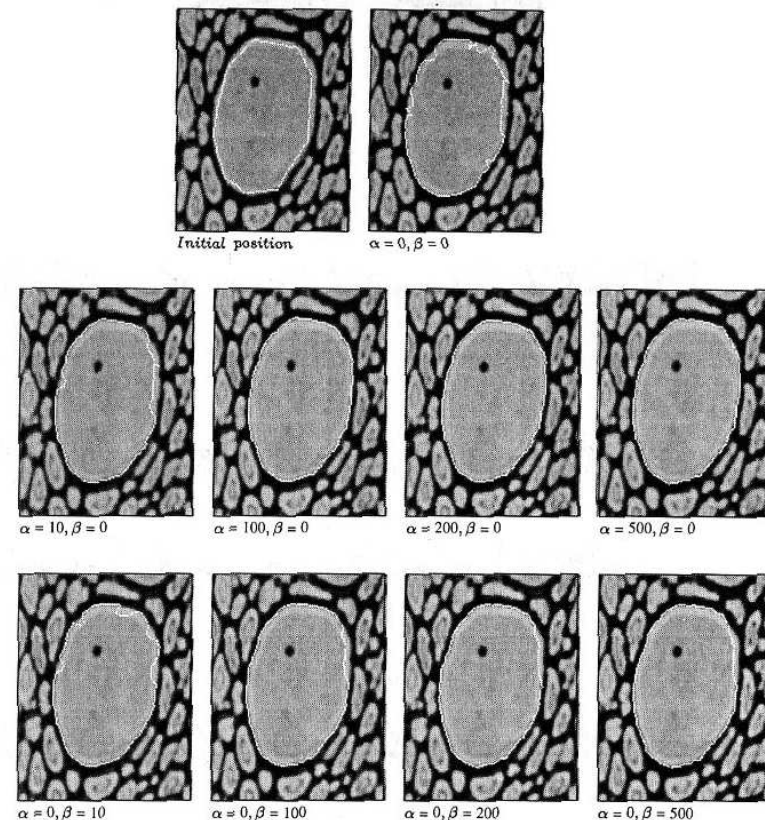
Modeli aktivnih kontur – kače

(11)

Primer: Segmentiranje mikroskopske slike lesa (vir: [5])



- ▷ Začetni položaj 1
- ▷ Variiranje parametrov α in β
- ▷ **Pravilne segmentacije ni bilo možno doseči!**



- ▷ Začetni položaj 2
- ▷ Variiranje parametrov α in β
- ▷ **Dosegli pravilno segmentacijo!**

- ▷ Možne naprednejše aplikacije:
 - ★ sledenje počasnemu gibanju objekta skozi zaporedje slik
 - ★ iskanje ujemanja v stereo slikah
 - ▷ Predlagan je dodatni energijski funkcional:

$$E_{stereo} = \left(\mathbf{v}^L(s) - \mathbf{v}^R(s) \right)^2$$

- ▷ Variante kač [4]:
 - ★ kače GVF (*gradient vector flow*)
 - ★ kače baloni (*balloon snake*)
 - ★ difuzijske kače (*diffusion snakes*)